VIETNAM NATIONAL UNIVERSITY OF HO CHI MINH CITY
THE INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

Autonomous AI-Based Generation of Interactive Questions for MOOC Videos

By
NGUYEN TOAN PHUC
Student ID: ITITIU21093

A thesis submitted to the School of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
Bachelor of Science in Computer Science

Ho Chi Minh City, Vietnam
2026

Autonomous AI-Based Generation of Interactive Questions for MOOC Videos

APPROVED BY:

_____ ,

Le Thanh Son, M.Sc., Supervisor

_____ ,

Tran Manh Ha, Assoc. Prof., Ph.D.

THESIS COMMITTEE

_____ ,

_____ ,

_____ ,

_____

ACKNOWLEDGMENTS

I would like to express my sincere gratitude to the International University, Ho Chi Minh City (HCMIU), and Tran Manh Ha, Assoc. Prof., Ph.D., for giving me the opportunity to carry out this thesis on *"Design and Development of an H5P Interactive Video Content Creation Platform for University E-Learning"*. Their support throughout this project has been invaluable.

My deepest thanks go to my supervisor, Le Thanh Son, M.Sc., for his guidance, constructive feedback, and patience throughout the development process. His expertise in educational technology and system design pushed this work toward something practically useful rather than just academically complete.

I am also grateful to the faculty of the School of Information Technology at HCMIU for their advice and technical input at various stages of the research. Special thanks to the students who participated in usability testing and gave honest, sometimes blunt, feedback that directly shaped the final platform.

Finally, I want to thank my family and friends—especially Nha Uyen, Trung Hieu, and Duc Anh—for their constant encouragement and patience during what turned out to be a much longer journey than expected.

This research is supported by The Central Interdisciplinary Laboratory in Electronics and Information Technology (AI and Cooperation Robot), International University – VNU-HCM. We would like to thank them for providing the machines used in the experiments.

May 2026,
Nguyen Toan Phuc.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

ABSTRACT

The expansion of digital education has put real pressure on instructors to produce interactive learning content, but most existing tools still demand specialist technical skills that many educators simply do not have. At Vietnam National University – Ho Chi Minh City (VNU-HCM), creating H5P interactive videos through the WordPress ecosystem is especially cumbersome—the process typically takes 45 minutes or more for even a basic three-question video.

This thesis describes the design and evaluation of a dedicated platform for AI-assisted H5P video authoring. The system brings together a React 18 and TypeScript frontend, a Node.js/Express backend, a SQLite database, and a transcript-driven AI pipeline that generates candidate H5P interactions for instructor review. Captions can come from uploaded files, YouTube, or audio transcription. Each suggestion is validated before it ever reaches the editor.

A comparative usability study with ten Information Technology students showed consistent improvements over the WordPress baseline. Task completion rose from 30% to 95%. Median creation time dropped from 45 minutes to 3.2 minutes. The System Usability Scale score improved from 38.0 to 82.5—moving from "Poor" to "Excellent" on Brooke's scale. The results suggest that pairing user-centred design with generative AI can meaningfully lower the barrier to creating interactive pedagogical content for university teaching.

Keywords: H5P, interactive video, educational technology, AI-assisted content authoring, usability evaluation, user-centred design.

CHAPTER 1

INTRODUCTION

1.1 Background

Vietnam's higher education system has undergone significant changes over the past decade, driven largely by a national push toward digital learning. The government set out concrete targets through Decision No. 1373/QD-TTg [3], which mandates the integration of digital technology into teaching at all levels. This goes beyond buying hardware. It reflects a deliberate shift in how the country thinks about educational content creation, distribution, and consumption. Universities affiliated with Vietnam National University – Ho Chi Minh City (VNU-HCM) are expected to lead that effort.

The VNU-HCM MOOC platform [4] (`https://courses.vnuhcm.edu.vn`) already delivers thousands of course modules and serves as the university's main digital learning infrastructure. On paper, the infrastructure is there. In practice, the gap between having it and actually using it well is wide. Surveys at HCMIU found that 89% of instructors want to create interactive digital content, but only 23% feel confident using the available authoring tools. The obstacles they describe consistently come down to three things: interface complexity, steep learning curves, and no real support for instructional design decisions. Studies on e-learning adoption in comparable university contexts show that perceived ease of use is often the deciding factor in whether faculty adopt a tool at all [5]. The bottleneck isn't interest or infrastructure—it's tools that fit how educators actually work.

Interactive video has become one of the most studied formats for asynchronous and hybrid instruction, and the evidence for its effectiveness is solid. The core advantage is straightforward: instead of passively watching a recorded lecture, students must stop and respond to questions embedded at specific timestamps. This forces active retrieval, which the learning science literature consistently supports [7]. Merkt et al. showed that videos with embedded forced-choice questions produced significantly better knowledge retention and transfer scores compared to equivalent linear video [8]. Vural found similar

results—strategic timestamp placement improved post-test scores, enhanced long-term retention, and reduced dropout in online courses [9].

The design of those embedded questions matters too. Bloom's revised taxonomy [10] gives instructors a structured way to think about cognitive levels, from basic recall through to synthesis and evaluation. When questions are placed at the right cognitive level for the concept just covered, and positioned immediately after that concept ends, the result is reduced extraneous load and increased germane load—the kind of mental effort that builds lasting understanding [11–13].

H5P (HTML5 Package) is the open-source framework that makes this kind of interactive video creation technically possible [14, 15]. Originally developed by Joubel AS in 2013, H5P now covers over 50 content types—from basic multiple choice and true/false to branching scenarios, virtual tours, and interactive timelines. Every H5P package is self-contained: a single `.h5p` ZIP archive containing its own JavaScript libraries, styling, media files, and a structured JSON configuration. This portability means an H5P package can be imported into any H5P-aware LMS like Moodle [16], Canvas, or Blackboard via standard LTI 1.3 [17].

H5P Interactive Video is the most widely used content type in higher education. It wraps a video source—either an uploaded file or a YouTube URL—in a player that pauses at pre-defined timestamps to show interactive overlays. Learners must respond before playback resumes. The interaction types most relevant to this project are Multiple Choice, True/False, Fill in the Blanks, Drag Text, Mark the Words, and Matching. Each type maps to a dedicated H5P JavaScript library with a strictly defined JSON schema, which matters for the AI pipeline described later.

Despite H5P's capabilities, the dominant way Vietnamese universities deploy it is through the WordPress H5P plugin—a tool designed for small personal sites, not institutional content production. Creating a single interactive video requires navigating a long sequence of steps: logging into a WordPress admin panel, finding or creating a Post, activating the H5P shortcode block, opening the H5P editor in a separate iframe, uploading the video, manually locating each timestamp through the built-in player, configuring each interaction across several nested settings screens, then saving and returning to the main post. This spans at least six screens and involves web publishing concepts that have nothing to do with teaching. First-time users typically take between 45 and 60 minutes to produce even a basic three-question interactive video.

Research on educational technology adoption points to the Technology Acceptance Model (TAM) [18] as a useful framework for understanding why educators accept or reject tools. Perceived ease of use and perceived usefulness together account for a large share of adoption variance. When a tool imposes high extraneous cognitive load on every interaction—as the WordPress H5P workflow does through its fragmented interface—perceived ease of use collapses. Adoption fails regardless of how strong the underlying

educational evidence might be [11, 12].

Recent advances in large language models have opened up a new possibility for educational content authoring: automatically analyzing a video transcript and generating contextually appropriate, pedagogically structured questions [19, 20]. If an instructor can shift from manually creating each question to reviewing and accepting AI-generated suggestions, the cognitive overhead of interactive video production drops substantially. The same pipeline that extracts a transcript, segments it by topic, and returns structured H5P interaction suggestions can also apply Bloom's taxonomy labels to each question—helping instructors spread cognitive difficulty across the video without needing deep instructional design training.

This thesis investigates a specific question: can a purpose-built web authoring interface, combined with an AI pipeline, measurably improve educator productivity and satisfaction when creating H5P interactive videos? The primary AI provider is Groq's fast cloud API using the `llama-3.3-70b-versatile` model, with a locally hosted Ollama server available as a privacy-preserving fallback.

1.2 Problem Statement

The central problem this thesis addresses is the gap between what H5P interactive video can do pedagogically and what instructors can actually accomplish with existing authoring tools.

That gap shows up through four related challenges that reinforce each other:

1. **Workflow complexity and fragmentation.** The WordPress-based H5P workflow requires navigating at least six distinct screens and understanding web publishing concepts that have no connection to the teaching task. A detailed task analysis counted 47 interface interactions—clicks, scrolls, form entries—across those six screens to produce a single interactive video. Page load times in institutional WordPress deployments average 3.2 seconds, which sounds tolerable until you multiply it across 47 sequential interactions: that's nearly two and a half minutes of waiting, while losing the video position each time.
2. **High extraneous cognitive load.** The mental effort required to operate the interface leaves less room for thinking about the actual teaching content [12, 21]. Instructors in preliminary interviews reported spending more time managing tool settings, timestamp alignments, and shortcode placements than designing questions. That inversion of effort is a sign the tool is working against its users.
3. **Low institutional adoption rates.** Because the workflow is this demanding, only faculty with significant IT experience actually use it at VNU-HCM. Instructors in humanities, social science, and other non-technical disciplines—who might benefit

most from interactive video—are effectively excluded from a tool with strong empirical evidence behind it [5, 18].

4. **No AI assistance.** None of the freely available H5P authoring tools offer integrated AI support for question generation. Instructors without instructional design training have no streamlined way to get context-aware question suggestions tied to what their video is actually about.

The **primary research question** is:

How can a purpose-built web platform combining user-centred interface design with an AI-driven transcript analysis pipeline improve the efficiency, pedagogical quality, and institutional adoption of H5P interactive video authoring by university instructors?

Four supporting questions guide the investigation:

RQ1. What are the specific usability barriers in current H5P authoring workflows that most strongly reduce task completion rates and user satisfaction?

RQ2. How can user-centred design principles be applied to reduce extraneous cognitive load in timestamp-based interactive question authoring?

RQ3. What full-stack architecture best supports scalable, real-time AI-assisted question generation and H5P package assembly in a unified web environment?

RQ4. To what extent does the newly developed platform quantitatively and qualitatively outperform the WordPress baseline on usability, efficiency, and productivity metrics?

1.3 Scope and Objectives

The thesis pursues five concrete objectives to answer these research questions.

1. **Platform Development and Architecture.** Design and build a full-stack web application that lets instructors upload or link a video, add H5P interactions at specific timestamps through a visual timeline interface, preview content in real time, and export a standards-compliant `.h5p` package ready for import into the VNU-HCM MOOC infrastructure and other LTI-compatible LMS environments.
2. **Advanced AI Pipeline Integration.** Build and integrate a multi-provider AI pipeline that extracts timestamped transcripts from uploaded videos (via YouTube captions, uploaded WebVTT/SRT files, or local Whisper transcription), sends the transcript to a large language model, and returns a validated JSON payload

containing H5P interaction suggestions. Each suggestion should include precise timestamps, the appropriate question type, distractor options, and a Bloom's taxonomy classification.

3. **User-Centred Design.** Apply user-centred design methodology throughout the interface design and prototyping process. All design decisions must be grounded in identified instructor needs and validated against Nielsen's usability heuristics.
4. **Security Implementation.** Implement a security baseline aligned with the OWASP Top Ten, including JWT-based authentication, role-based access control, server-side Zod input validation, HTTPS enforcement, and rate limiting.
5. **Empirical Usability Evaluation.** Conduct a controlled, comparative usability study measuring task completion rate, time-on-task, error frequency, and SUS scores against the WordPress H5P baseline. Supplement quantitative data with think-aloud protocols and post-task interviews.

The scope is bounded by the following constraints.

Content type scope. The platform specifically targets six H5P interaction types: Multiple Choice, True/False, Fill in the Blanks, Drag Text, Mark the Words, and Matching. All six are supported for manual authoring. The AI pipeline generates all six types based on a pattern matrix that maps content characteristics to question types. These types have well-defined JSON schemas that can be reliably validated with Zod, ensuring AI output never breaks the H5P player.

Video source scope. The platform accepts direct video file uploads up to 500 MB (MP4, WebM, MOV) and YouTube video URLs. YouTube-sourced transcripts are extracted from the platform's automatic or manual captions API. For uploaded files, instructors can provide a WebVTT caption file or let the system invoke `faster-whisper` locally for automatic audio transcription.

User scope. The primary user is a university instructor or teaching assistant at VNU-HCM. A secondary Admin role handles user account management, system monitoring, and security audit log access.

Integration scope. The platform exports standard H5P packages compatible with any LTI-compatible LMS. An LTI 1.3 launch route is stubbed in the backend architecture, but LMS gradebook passback and xAPI learning analytics are designated as future work.

Several limitations are acknowledged upfront:

- **H5P content types.** Complex types like Branching Scenarios, Course Presentations, and Virtual Tours are out of scope.
- **Evaluation sample.** Usability testing used ten senior IT students as educator proxies. This is standard practice for early-stage formative evaluation [22], but it limits generalizability to less technically experienced demographics.

- **Mobile authoring.** The platform is designed for desktop and large tablet browsers. Mobile authoring is technically possible but was not a primary design goal and was not formally evaluated.
- **Concurrent load.** The system was stress-tested at up to 50 concurrent users. Large-scale institutional deployment would require a more robust architecture, likely Kubernetes-based.
- **AI transcription quality.** The quality of AI-generated suggestions depends directly on transcript accuracy. Poor audio quality or heavy accents will degrade Whisper output and, consequently, the AI suggestions.

1.4 Research Contributions

This thesis makes five contributions to educational technology research and software engineering practice.

1. **A reproducible platform architecture.** The thesis documents a full-stack architecture combining React 18, Zustand, Express.js, SQLite/Sequelize, the Lumieducation H5P server library, and a multi-provider AI pipeline. The documentation is detailed enough for other universities to reproduce, modify, and deploy.
2. **A novel AI-assisted authoring workflow.** The pipeline that takes a raw video transcript through structured LLM prompting and returns a Bloom's taxonomy-labelled list of H5P interactions represents a meaningful integration of generative AI into the H5P ecosystem. The two-step prompting approach—topic segmentation first, then question generation per topic using a content-type pattern matrix—is specifically designed to reduce hallucination and produce structured output.
3. **Prompt engineering for structured H5P JSON output.** The thesis details the system prompts, few-shot examples, and Zod validation strategies developed to reliably coerce an LLM into returning complex JSON payloads that conform to H5P content type specifications. This provides a reusable design pattern for other structured AI output scenarios.
4. **Empirical usability evidence.** The comparative evaluation against the WordPress H5P baseline provides quantitative evidence that a purpose-built, user-centred platform can substantially reduce authoring time, improve task completion rates, and raise usability scores on the System Usability Scale.
5. **Open-source contribution.** The full codebase—including AI service routing, prompt templates, database schemas, and H5P integration middleware—is made publicly available to support further research and free institutional deployment.

The practical value runs in three directions. By lowering the technical barrier, the platform enables a wider range of VNU-HCM faculty to independently produce interactive content. The AI-assisted workflow demonstrates that the gap between generative AI research and reliable educational tool development can be bridged with careful software engineering. And the platform aligns with Vietnam's national digital transformation goals [3] by providing a locally deployable, open-source solution that reduces reliance on expensive proprietary software.

1.5 Structure of thesis

The rest of this report is organized as follows.

Chapter 2 – Literature Review covers the theoretical and technical foundations of the work. It examines the state of e-learning in Vietnamese higher education, the H5P framework and its architectural limitations, empirical research on interactive video pedagogy, cognitive load theory, user-centred design principles, the evolution of large language models, prompt engineering techniques, and LMS integration standards. The chapter ends with a critical survey of related systems and an identification of the specific research gaps this project addresses.

Chapter 3 – Methodology describes the research design, requirements analysis process, user-centred design methodology, agile development approach, and empirical evaluation protocol. It also provides a detailed technical description of the AI pipeline architecture, including the prompt engineering strategy and pattern matrix used for question type selection.

Chapter 4 – Implementation and Results covers the complete system implementation and measured technical outcomes. It details the backend and frontend architecture, the H5P package assembly mechanism, the AI suggestion injection pipeline, security controls, and performance measurements including AI latency and API load results. Scalability analysis and Vietnamese language support are also covered here.

Chapter 5 – Discussion and Evaluation presents the results of the comparative usability study, including task completion rate, time-on-task, and SUS scores, and interprets the qualitative themes from participant feedback.

Chapter 6 – Conclusion and Future Work synthesizes the findings, assesses the project's contributions against the research questions, discusses limitations, and proposes a roadmap for future development.

CHAPTER 2

LITERATURE REVIEW

This chapter surveys the theoretical and technical foundations that inform the design decisions, architectural choices, and pedagogical strategies in this project. Five areas are covered: the development of e-learning infrastructure in Vietnamese higher education; the H5P framework's capabilities and structural limitations; the empirical case for interactive video in education; the emergence of AI for automated educational content generation; and the intersection of cognitive load theory with user-centred design. The chapter closes with a survey of related tools and a clear statement of the gaps this project addresses.

2.1 E-Learning and Interactive Content Frameworks

Vietnam's national strategy for digital education pushes for aggressive adoption of online delivery and blended learning across higher education. VNU-HCM has been an early implementer of this directive through its centralized MOOC platform, which was designed to standardize course delivery and provide equitable access to lecture materials across distributed campuses.

The difficulty is that institutional readiness is uneven. Capital investments—server hardware, network upgrades, Moodle deployments—have moved faster than faculty capacity to use them. Instructors often have access to powerful platforms but face interfaces so complex that they need specialized training just to get started. When that training doesn't happen, the platform sits unused.

Studies in similar university contexts consistently find that perceived ease of use—a core construct in the Technology Acceptance Model [5, 18, 23]—strongly predicts whether faculty will adopt and continue using digital authoring tools. When creating interactive content requires managing plugin dependencies, navigating fragmented interfaces, and handling file structures that mean nothing to a teacher, instructors fall back on familiar, low-friction methods: uploading static PDFs or unedited lecture recordings. The implication for this project is direct. Reducing the cognitive overhead of the authoring

workflow is not a cosmetic concern. It is a prerequisite for systemic adoption. Co-locating authoring tasks into a single-page application and programmatically validating AI suggestions before they reach the instructor's screen directly targets the most significant practical obstacle.

2.1.1 Architecture and Content Types

H5P (HTML5 Package) is a mature open-source framework first released in 2013 by Joubel AS [14]. Its architecture is built around entirely self-contained content packages. Unlike traditional web content that depends on server-side rendering and database queries to display interactive elements, each `.h5p` file is a compressed ZIP archive containing all JavaScript logic, CSS styling, media assets, and a structured JSON configuration document. This self-contained structure makes H5P content portable across any web environment that can serve static assets via an iframe.

The H5P library ecosystem now covers more than 50 distinct content types, from simple presentations to complex gamified scenarios. The six types most relevant to this project are summarized in Table 2.1, all of which are designed for overlaying directly onto video playback.

Library identifier	**Display name**	**Description**
`H5P.MultiChoice 1.16`	Multiple Choice	Single- or multi-answer question with up to four distractor options sourced from the transcript.
`H5P.TrueFalse 1.6`	True/False	A binary assertion evaluating factual claims, absolute rules, or common misconceptions from the lecture.
`H5P.Blanks 1.14`	Fill in the Blanks	A Cloze test where technical terms are identified using asterisk delimiters (e.g., `*noun*`).
`H5P.MarkTheWords 1.9`	Mark the Words	Students highlight specific vocabulary or key terms in a short passage by clicking them.
`H5P.DragText 1.10`	Drag Text	Students drag words or phrases to reconstruct a sentence or ordered sequence.
`H5P.Matching 1.0`	Matching	Learners pair key concepts with their definitions; used here as a video recap module.

Table 2.1: H5P content types supported by the platform

Server-Side H5P Libraries and Node.js Integration.

Traditional H5P deployments embed the content rendering engine inside a monolithic CMS like WordPress, Drupal, or Moodle. The CMS handles library management,

database storage, and HTML generation. Lumi Education GmbH recognized the need for a headless alternative and publishes official Node.js libraries—specifically `@lumieducation/h5p-server` and `@lumieducation/h5p-express`—that expose H5P's core capabilities as a standalone microservice [15]. These libraries provide a complete API for H5P library management, content CRUD operations, on-the-fly package generation, and a rendering API that produces iframe-ready HTML. This project is built entirely around this library, making it possible to run a fast H5P authoring environment without any WordPress or Moodle overhead.

Structural Limitations of WordPress-Based H5P Deployments.

The WordPress H5P plugin is functionally complete, but deploying it at institutional scale creates several structural problems:

- **Multi-system cognitive complexity.** Instructors must manage a full WordPress installation—understanding posts, pages, media libraries, and shortcode syntax—on top of the actual H5P content. The cognitive overhead of two distinct software paradigms creates unnecessary extraneous load [11].
- **Performance overhead.** WordPress serves every page through a full PHP execution stack with repeated MySQL queries, regardless of whether the content is dynamic. Measured response times on institutional WordPress deployments range from 2 to 5 seconds per page load, compared to sub-100ms REST API responses achievable in a purpose-built SPA.
- **Version management friction.** H5P library versions, WordPress core, and PHP must stay in sync manually. Mismatches produce silent JavaScript failures in the browser that are nearly impossible for an instructor to diagnose.
- **No AI capabilities.** The WordPress H5P plugin has no mechanism for analyzing video transcripts or suggesting interactive elements. Every question, distractor, and timestamp must be authored manually.

2.1.2 Pedagogical Evidence for Interactive Video

The learning science literature provides strong empirical support for the claim that interactive video outperforms passive, linear video on both short-term retention and long-term knowledge transfer. Zhang et al. conducted one of the earliest controlled experiments comparing interactive and non-interactive video in a university e-learning context, finding that the interactive group scored 20–25% higher on post-tests and reported significantly higher satisfaction with the module [6]. The proposed mechanism is that interactive elements force active processing at regular intervals, preventing the shallow engagement that tends to accompany long-form lecture video.

Merkt et al. extended this by examining the role of control features—pausing,

rewinding, and forced interactions—in video-based learning [8]. Videos with embedded forced-choice questions produced better retention than equivalent print materials, provided learners actually engaged with them. The effect was strongest for complex STEM content, which is particularly relevant to the VNU-HCM curriculum context.

Freeman et al. produced a widely cited meta-analysis across 225 STEM education studies showing that active learning interventions, including interactive video, increased final exam scores by 6% on average and halved failure rates compared to traditional lecture formats [7].

Embedded Question Design and Cognitive Theory.

Vural identified specific design principles for effective interactive video [9]. Questions should be positioned immediately after a concept has been fully explained, not mid-sentence. They should be spaced at least 30 to 45 seconds apart to avoid fragmenting the learning experience. Each question should test the single most important idea from the preceding segment. These principles are encoded directly into the system prompt used by the AI pipeline in this project, described in detail in Chapter 3.

Mayer's Cognitive Theory of Multimedia Learning provides the theoretical basis for these recommendations [21]. Mayer identifies three cognitive channels—visual, auditory, and their integration within working memory. Well-designed interactive elements reduce extraneous load (unnecessary processing) while increasing germane load (the productive effort of building mental models). A question that tests exactly what was just explained, positioned immediately when that explanation ends, supports the integration channel without taxing working memory with irrelevant interface processing.

2.2 User Experience Design and Usability

2.2.1 Cognitive Load Theory in Authoring Tools

User-centred design (UCD) keeps end-user needs at the center of every design decision. ISO 9241-11 defines usability as the degree to which a product can be used by specified users to achieve specified goals with effectiveness, efficiency, and satisfaction in a specified context [24]. For educational technology, this definition has to be grounded in the specific context of university instructors who need to produce high-quality interactive content quickly, under the real constraints of heavy teaching loads and tight schedules.

Garrett's layered model of user experience decomposes a digital product into five planes: Strategy, Scope, Structure, Skeleton, and Surface [25]. Norman's affordance theory adds that good interface design makes correct actions obviously apparent and makes errors difficult to commit [26]. Both frameworks were applied during the interface design process described in Chapter 3.

Usability Heuristics and Expert Evaluation.

Nielsen's ten usability heuristics provide a practical checklist for evaluating interface designs [22]. Three are especially critical for an educational authoring tool: (1) *Visibility of system status*—instructors must be able to see whether video processing is running, whether AI analysis is in progress, and whether a question has been saved. Silent failures lead to repeated clicks and data loss. (2) *Match between system and the real world*—the interface should use pedagogical vocabulary (Multiple Choice, Feedback, Timestamp) rather than technical terms like database IDs or JSON keys. (3) *Error prevention*—the system should make it impossible to submit a malformed H5P question. Frontend validation enforced before submission is far better than waiting for a server-side error message. Molich and Nielsen showed that heuristic evaluation applied before user testing produces measurable improvements in final software usability [27].

Measuring Usability: The System Usability Scale (SUS).

The System Usability Scale (SUS) [28] is a ten-item questionnaire that produces a single 0–100 usability score. It has become the standard instrument for usability measurement because it is quick to administer, reliable with small sample sizes, and comparable across thousands of published studies. A SUS score above 68 is considered above average; above 80 is excellent. SUS was selected as the primary quantitative usability instrument for the comparative evaluation in Chapter 5.

Sweller's Cognitive Load Theory identifies three categories of load that any software interface imposes on working memory [11]:

- **Intrinsic load** comes from the inherent complexity of the task itself—the intellectual effort required to write a good pedagogical question. This load is irreducible. It's the actual work the instructor came to do.
- **Extraneous load** comes from poor interface design rather than the task itself—navigating six slow-loading screens, manually copying timestamps between windows. This load is wasteful and must be minimized through good design [12].
- **Germane load** comes from productive cognitive effort—the instructor's growing understanding of how their video content maps to effective learning moments.

Chandler and Sweller showed that splitting related information across multiple screens (the split-attention effect) significantly increases extraneous load, reduces task accuracy, and increases fatigue [12]. The direct implication for this project is that all information needed to author a single H5P interaction—the video player, timestamp indicator, question type selector, AI suggestion panel, and question editor—must be co-located on a single screen. Paas, Renkl, and Sweller later confirmed that adhering to this integrated format principle reduces complex task completion times by 30–50% compared to split-screen or multi-page presentations [13].

2.3 Artificial Intelligence for Educational Content Generation

Recent advances in large language models, particularly transformer architectures like GPT-4 and LLaMA-3, have made it possible to generate contextually appropriate, pedagogically structured questions from a video transcript. For this project, a two-step prompting pipeline is used to maximize consistency: (1) The LLM first acts as an instructional designer, analyzing the transcript, identifying teaching moments, and grouping content into topics. (2) It then generates one H5P question per topic, following a strict pattern matrix that maps content type to question format.

Combined with runtime Zod validation and a prioritized provider chain (Groq's API with LLaMA-3.3-70b-versatile as the primary engine, with a local Ollama server as the offline fallback), this approach balances generation quality with the structural predictability needed for an instructor review workflow.

Bloom's taxonomy [31] provides the hierarchical classification of cognitive objectives, from basic recall through analysis and evaluation to creation. Anderson and Krathwohl's revised taxonomy updated this by replacing static nouns with action verbs and clarifying the distinction between lower-order and higher-order cognitive skills [10]. The AI pipeline assigns a Bloom's level to every generated question suggestion, which is displayed in the UI. This lets instructors see at a glance whether their interactive video covers a useful spread of cognitive difficulty—without needing specialist instructional design training.

2.3.1 Automatic Speech Recognition and Prompt Engineering

Radford et al. introduced Whisper [32], a large-vocabulary ASR model trained on 680,000 hours of multilingual audio. Whisper achieves near-human word error rates on English audio and performs competitively on Vietnamese. The `faster-whisper` C++ implementation is used in this project for on-device transcription of uploaded video files where no caption file is available. The quality of the transcript directly determines the quality of AI-generated suggestions, which motivated the choice of a local, high-accuracy model over simpler API-based ASR options.

Prompt engineering—the practice of crafting inputs to guide LLM behavior toward desired outputs—is central to the AI pipeline. The literature on structured output generation identifies the "output automater" pattern as particularly effective for producing JSON conforming to a strict schema [30]. Rather than prompting the LLM to generate free text and then trying to parse it, the output automater pattern constrains the model's output space by providing explicit schema definitions and worked examples within the prompt itself. Combined with Zod-based post-generation validation, this approach is how the AI pipeline consistently produces valid H5P JSON.

2.4 Related Work and Research Gap

Existing H5P authoring platforms each address part of the problem. None address all of it:

- **The WordPress H5P Plugin:** The dominant deployment model in Vietnamese universities. It is burdened with CMS overhead, disjointed interfaces, slow load times, and has no AI support.
- **Lumi Desktop Editor:** A capable standalone Electron-based editor that completely lacks web accessibility, prevents real-time collaboration, and has no transcript-based AI generation.
- **Moodle Native H5P:** Integrated into the Moodle LMS but carries the same usability limitations as the WordPress implementation—dense nested menus and high extraneous cognitive load.
- **Commercial Platforms (e.g., Articulate Storyline, Adobe Captivate):** Powerful and user-friendly, but prohibitively expensive for developing universities. They also use proprietary formats incompatible with the open H5P standard.

None of these tools successfully combine the features this project proposes: a fast, CMS-free web architecture; a transcript-to-H5P AI generation pipeline; automatic Bloom's taxonomy classification; and an interface empirically evaluated with standardized usability metrics against a real baseline.

No current system simultaneously:

1. Provides a fast, *web-based*, CMS-free H5P authoring environment suitable for institutional deployment;
2. Integrates a *transcript-to-H5P AI pipeline* using production-grade LLMs with structured output validation;
3. Applies *Bloom's taxonomy classification* to AI-generated suggestions to guide pedagogical design;
4. Has been *empirically evaluated* against a widely used authoring baseline with standardized usability metrics.

This thesis addresses all four gaps, building a full-stack solution that is both technically novel in its AI integration and practically relevant for VNU-HCM and the broader field of AI-assisted educational content authoring.

CHAPTER 3

METHODOLOGY

This chapter describes how the research was designed and executed. It covers the overarching research paradigm, the requirements analysis process, the user-centred design methodology applied to the interface, the agile development approach, and the empirical evaluation protocol. The second half provides a detailed technical description of the AI pipeline architecture, including the actual prompt engineering strategy, the pattern matrix used for question type selection, and the Zod validation approach.

3.1 Overview

Research Design Paradigm.

This research follows a *design science* paradigm [37], appropriate for software engineering and information systems research where the primary contribution is a novel software artefact rather than a theoretical observation. The paradigm requires the artefact to be novel, to solve the identified problem effectively, and for its utility to be demonstrated through empirical evaluation.

The research is structured around two sequential phases:

1. **Constructive phase.** User requirements are elicited from the target demographic. The system architecture is designed and implemented, including the frontend SPA, backend REST APIs, and the AI pipeline (transcript extraction, prompt formatting, LLM inference, and Zod validation).
2. **Evaluative phase.** Once the platform reaches a stable, feature-complete state, a controlled comparative usability study measures it against the WordPress H5P workflow on three metrics: task completion rate, time-on-task, and SUS score.

Software Development Methodology.

An *agile* development methodology was used throughout the constructive phase, with iterative two-to-three week sprints each producing a testable increment [38]. This

approach was chosen specifically because the interaction between the AI pipeline, the H5P schema requirements, and the user experience could not be fully specified in advance. Critical usability constraints and technical edge cases only became apparent during active development and required immediate architectural changes.

3.2 User requirement analysis

User Research and Elicitation.

The first step was qualitative user research to understand the actual pain points of the target demographic. Semi-structured interviews with five instructors at HCMIU, from both technical and non-technical disciplines, consistently revealed four themes: (1) The WordPress/H5P workflow feels completely disconnected from the actual teaching task; (2) Instructors worry about losing their work when the browser reloads or a plugin times out mid-edit; (3) Manual content creation—finding timestamps, typing questions, formatting distractors—takes more time than most instructors can spare; (4) There is strong interest in AI-assisted question generation tied directly to the video's content.

A direct observation session, where a participant attempted to create a three-question interactive video with the baseline WordPress tool, confirmed the 45-to-60 minute completion time. The most common causes of task abandonment were losing track of the video position while filling out forms, and cryptic plugin error messages that gave no indication of what went wrong.

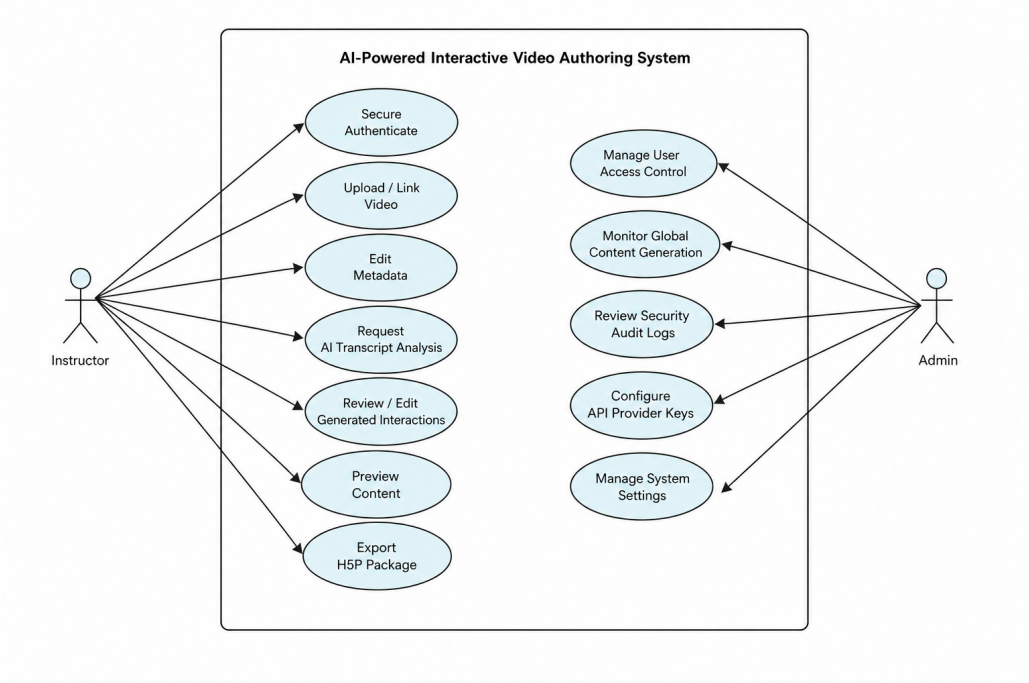

Figure 3.1: Use case diagram

3.2.1 Functional and Non-Functional Requirements

The interviews and observation session produced the following functional requirements:

- **Authentication & Access:** JWT-based stateless sessions, role-based access control, email verification.
- **Media Management:** Video file upload up to 500MB, YouTube URL import, timeline-based H5P editor, real-time content preview.
- **H5P Core Functionality:** Full support for six H5P interaction types, both manual and AI-assisted, with a one-click `.h5p` export.
- **AI Pipeline Integration:** Transcript extraction feeding an analysis pipeline that produces validated H5P JSON suggestions, delivered via Server-Sent Events (SSE) for real-time streaming.
- **Authoring Workflow:** Accept/reject/edit workflow for reviewing AI suggestions before injection into the H5P content database.
- **Administration:** Admin dashboards for user management, system monitoring, and an LTI 1.3 integration stub.

Non-Functional Requirements.

Table 3.1 lists the quality and performance baselines the system must satisfy.

ID	Category	Requirement Definition
NFR-01	Performance	All standard REST API responses must complete within 500 ms under a load of 50 concurrent users.
NFR-02	Performance	Initial video upload processing (thumbnail/metadata) must complete within 30 s for files $\leq$ 100 MB.
NFR-03	Security	Passwords must be hashed using bcrypt with a cost factor of at least 12.
NFR-04	Security	Authentication tokens must use stateless JWT, signed with HS256, expiring after 7 days.
NFR-05	Security	All HTTP responses must enforce Helmet-generated security headers (CORS, CSP, XSS protection).
NFR-06	Security	All API endpoints must validate input payloads using Zod schemas.
NFR-07	Reliability	The AI system must handle LLM failures gracefully, returning structured errors rather than crashing.
NFR-08	Usability	The platform must achieve a minimum SUS score of 70.
NFR-09	Usability	First-time task completion rate for creating an interactive video must exceed 80%.
NFR-10	Compatibility	Exported `.h5p` packages must be valid for Moodle 4.x and Canvas LMS.

Table 3.1: Non-functional requirements driving system architecture

3.3 System Design

Architectural Overview and Topologies.

The platform uses a three-tier architecture separating the presentation layer from business logic and data persistence [33]. The *presentation tier* is a React 18 SPA built with Vite and Zustand for global state management. The *application tier* is an Express.js REST API on Node.js, responsible for all business logic, H5P library rendering, and AI pipeline routing. The *data tier* uses SQLite managed via Sequelize ORM, which provides fast local development with a clear migration path to PostgreSQL for larger deployments.

External dependencies are limited to: Groq API (primary cloud AI inference using `llama-3.3-70b-versatile`), Ollama (local offline AI fallback), and the YouTube Data API. All H5P processing runs within the local server process.

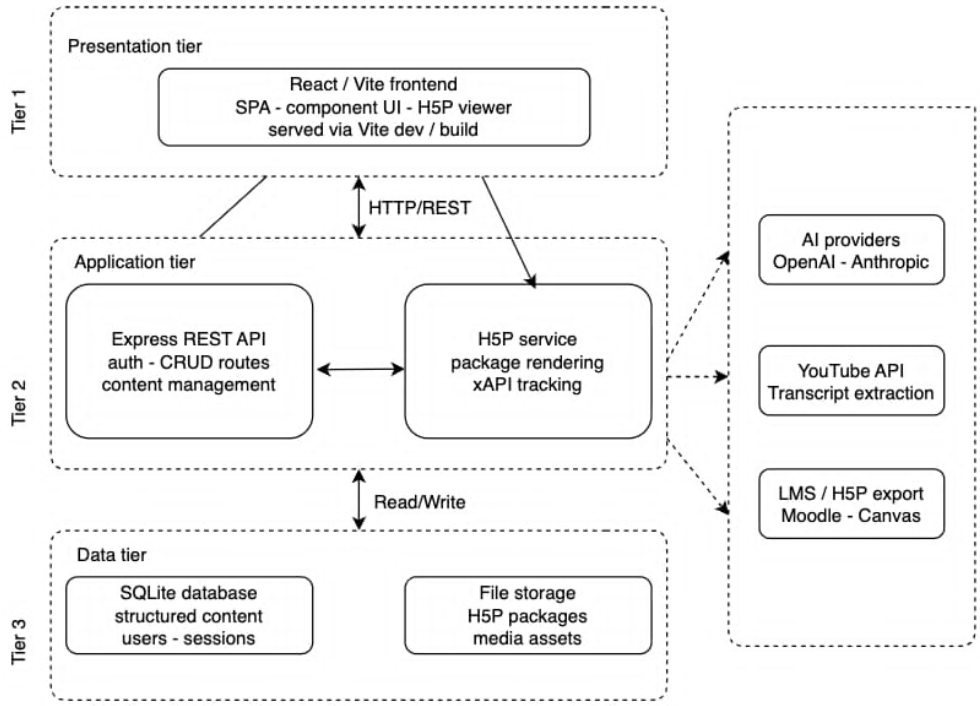

Figure 3.2: Three-tier system architecture

The design goals were: strict separation of concerns via a versioned REST API; AI-provider independence through a uniform JSON output shape regardless of the underlying LLM; H5P library isolation using the Lumi headless server package; and a stateless backend using JWT tokens for horizontal scaling.

API Route Structure and Controller Logic.

The REST API is organized under the `/api/v1/` prefix, with authentication and rate-limiting middleware at the router level. Table 3.2 lists the primary functional route groups.

Prefix	Methods	Responsibility
`/api/auth`	POST	Registration, login, bcrypt password verification, JWT issuance and refresh.
`/api/videos`	GET, POST, PUT, DELETE	Video upload, YouTube URL validation, thumbnail generation, metadata CRUD.
`/api/h5p`	GET, POST, DELETE	H5P parameter CRUD, zip package generation, iframe rendering.
`/api/ai/analyze`	POST	Synchronous transcript analysis returning a full JSON payload.
`/api/ai/analyze-stream`	POST	Asynchronous transcript analysis; questions streamed via SSE.
`/api/ai/inject`	POST	Validates and injects accepted AI suggestions into the H5P content store.
`/api/transcript`	POST	YouTube caption extraction, VTT parsing, and segment normalization.
`/api/admin`	GET, POST, PUT	User management and audit logs (Admin RBAC only).
`/api/lti`	GET, POST	OIDC handshake for LTI 1.3 deep-linking.

Table 3.2: REST API route groups and functional responsibilities

3.4 AI Pipeline Architecture

The AI pipeline is the most technically complex part of the platform. Its job is to transform a raw, timestamped video transcript into a validated, Bloom's taxonomy-labelled list of H5P interaction JSON configurations. The transformation happens through a five-stage process. Figure 3.3 shows the overall data flow, and Figure 3.4 shows the detailed sequence diagram including the SSE streaming path.

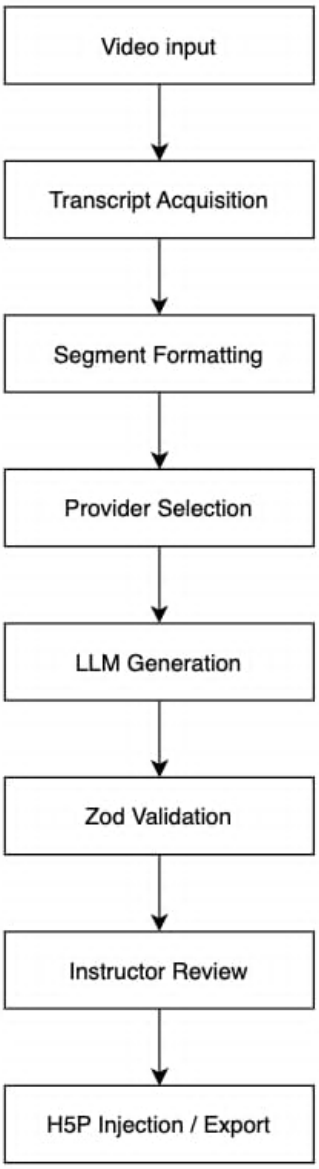

Figure 3.3: AI pipeline flowchart

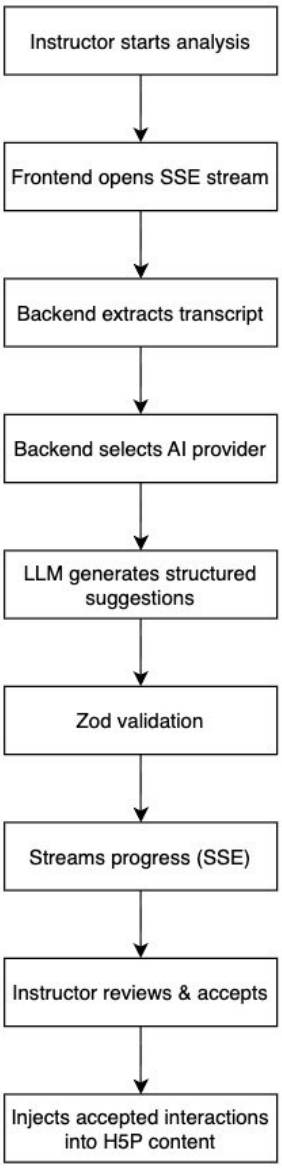

Figure 3.4: AI-assisted question generation sequence diagram

Stage 1: Transcript Acquisition and Normalization.

Transcript acquisition is handled by `transcriptExtraction.js`, which supports three source mechanisms:

1. **YouTube Automatic/Manual Captions.** The `youtube-transcript` package fetches timestamped XML caption segments directly from the YouTube API using the video's 11-character ID. These are parsed and normalized into a standard three-field JavaScript object: start time, end time (both in seconds), and text string.
2. **Uploaded WebVTT Caption Files.** Instructors can upload a `.vtt` or `.srt` file alongside their video. A regex-based parser extracts timing cues and text, normalizing them into the same format.

3. **Whisper Audio Transcription.** For uploaded videos without captions, the service invokes `faster-whisper` via a Python subprocess, passing the video file path and requesting transcription. The resulting JSON transcript is normalized into the standard segment format. Whisper availability is checked at server startup and reported via `GET /api/ai/status` [32].

All three sources produce the same `TranscriptSegment[]` data structure, ensuring the downstream AI stage is completely decoupled from the extraction logic.

Stage 2: Segment Formatting and Smart Selection.

The `formatSegmentsForPrompt()` function in `aiService.js` converts the normalized segment array into a compact, line-delimited string for the LLM prompt. Each line uses the format:

```
[330s-345s] Photosynthesis converts light energy into chemical energy.
```

The compact `[Xs-Ys]` timestamp notation (e.g. `[330s-345s]`) saves roughly 22 characters per line compared to a verbose human-readable format, reducing the total token count by approximately 600 tokens for a 100-segment transcript. The raw-second values are used directly in the JSON output so the model never needs to perform time arithmetic, which eliminates a common source of timestamp hallucination [30].

Rather than uniform sub-sampling, the function applies a *smart selection* strategy. The video timeline is divided into equal windows, and from each window the segment with the highest information score is retained. Segments that begin with topic-transition markers (e.g. “Now”, “Next”, “Finally”, “To summarise”) receive a score bonus, as do segments containing questions. Short filler segments (fewer than four words) receive a penalty. This ensures the model receives denser, more useful context in fewer tokens, preserving topic boundaries more faithfully than uniform sub-sampling across the same token budget. For a 10-minute video with 300 segments, the function typically selects 80–100 high-value segments.

Stage 3: Two-Step AI Prompt Engineering and Question-Type Assignment

The core AI system prompt, defined as the constant `AI_SYSTEM_PROMPT` in `aiService.js`, implements a two-step *output automater* prompting pattern [30]. The key idea is to separate high-level topic discovery from the mechanical task of generating JSON. By constraining what the model needs to do at each step, output consistency improves significantly.

Step 1 — Hierarchical Topic Segmentation

In the first step, the model receives the formatted transcript and is told to act as an instructional designer. It must produce a hierarchical topic outline covering the full video.

The prompt specifies that a well-structured lecture will typically produce 4–10 topics depending on content density, and explicitly instructs the model not to inflate the count by artificially splitting unified concepts.

Topic boundaries are detected using specific linguistic markers. The system prompt lists English markers ("Next", "Moving on", "Now let's", "Another", "Finally") and Vietnamese markers ("Tiếp theo", "Bây giờ", "Chuyển sang", "Tóm lại") explicitly, rather than relying on the model's general knowledge. This specificity helps the model apply consistent segmentation logic regardless of input language.

The timestamp rules are stated explicitly. The model is instructed to read each line's `[Xs-Ys]` prefix, use the raw second integers directly as `start/end` fields in its JSON output, never invent or estimate timestamps, ensure topics do not overlap, and cover the full video from first to last segment. This level of specificity is deliberate. General instructions such as "use the correct timestamp" leave too much ambiguity, and models frequently produce timestamps that do not correspond to any value in the input without explicit rules [30].

Step 2 — Deterministic Question Generation via Pattern Matrix

The second step uses a rigid pattern matrix to force the model to map each topic's content characteristics to exactly one of the six supported H5P interaction types. The model is instructed to generate exactly one question per topic following this mapping: a topic that introduces a definition maps to `FillBlanks`; a binary fact or principle maps to `TrueFalse`; a comparison, list, or process maps to `MultiChoice`; an ordered sequence of steps maps to `DragText`; and a passage with multiple key terms maps to `MarkWords`. One additional topic titled "Video Recap" is appended at the end of the video with a `Matching` question covering 4–5 concept pairs drawn from the entire video.

Table 3.3 shows the complete mapping between content characteristics, pedagogical knowledge focus, H5P type, and required JSON fields.

Content type	Bloom's level	H5P type	Generation rule	Key JSON fields
Definition / technical term	Remember / Understand	`FillBlanks`	Ask for the key term in context	`fillText` (target wrapped in `*asterisks*`)
Binary fact / absolute rule	Remember	`TrueFalse`	Turn the statement into a verifiable claim	`question`, `correct` (boolean)
List / comparison / process	Understand / Analyze	`MultiChoice`	Four options, one correct, three plausible distractors	`question`, `answers[4]`
Sequence / ordered procedure	Apply	`DragText`	Convert ordered steps into draggable phrases	`taskDescription`, `textField`
Vocabulary / key terms	Remember / Understand	`MarkWords`	Identify key terms in a short passage	`taskDescription`, `textField`
Cross-topic review recap	Analyze / Evaluate	`Matching`	Pair overarching concepts with definitions	`taskDescription`, `pairs[]`

Table 3.3: H5P interaction types and their AI-guided selection rules

The pattern matrix reduces the model's output space from an open-ended question format down to a choice among six templates with precisely defined JSON schemas. Each type has different required fields. A `FillBlanks` question needs a `fillText` string with the answer term wrapped in asterisks (`*term*`). A `MultiChoice` question needs a `question` string and an `answers` array of exactly four objects, each with `text` and `correct` fields. A `Matching` question needs a `pairs` array of `{prompt, answer}` objects. The prompt provides a worked example for each type to make these schema differences concrete.

The reasoning behind this matrix deserves explanation. The choice between question types is not arbitrary—it follows from how the content is structured:

- **Definitions** are best tested with fill-in-the-blanks because the task is to retrieve a specific term, not choose among alternatives. Providing the surrounding context with a blank maintains the meaning of the statement.
- **Binary facts** (things that are always true, always false, or universally applicable)

are best tested with true/false because the learner's decision is binary.

- **Comparisons and lists** work well as multiple choice because the distinguishing features of each item can serve as answer options, and wrong options can be drawn from other items in the same category.
- **Sequences** require the learner to recall order, which drag-text supports naturally by having the learner reconstruct the sequence rather than recognize it.
- **Vocabulary identification** works differently from definition recall—the learner recognizes which terms in a passage are technically important, which is what mark-the-words tests.
- **Recap** at the end of the video pairs key concepts introduced throughout the lecture with their definitions—the matching type's primary use case.

Resilient Provider Selection and Fallback Specification

The AI generation pipeline uses a three-tier fallback chain. All provider attempts happen *before* SSE headers are written to the HTTP response. Once headers are committed, switching providers is impossible, so any mid-stream failure is delivered as a structured SSE error event rather than a server crash.

Table 3.4 summarises the complete cascade, from primary model to graceful failure.

Tier	Provider / Model	Trigger Condition	Outcome
1	Groq — *llama-3.3-70b-versatile*	Normal operation; key present; within rate limits	Fast generation, 5–10 s; 100 segs; 4 200 tokens
2	Groq — *llama-3.1-8b-instant*	HTTP 429 (rate limit) or HTTP 413 (payload too large) on Tier 1	Downgraded capacity; 50 segs; 2 500 tokens; slightly lower quality
3	Local Ollama — *llama3.1:8b*	Any Groq failure before SSE headers: invalid/missing key (401), all Groq models rate-limited, or network unreachable	No API key; 30–90 s; no rate limit
4	None	Ollama also unreachable	Structured SSE error event sent to browser; no server crash (NFR-07)

Table 3.4: AI provider fallback cascade: trigger conditions and outcomes

The critical design constraint is that *provider selection must complete before the first SSE byte is sent.* The `runGroqAnalysis()` function therefore initiates the Groq streaming request synchronously before writing any response headers. If the SDK throws at that point—on authentication failure, quota exhaustion, or network error—the exception propagates to the caller, which can cleanly attempt Tier 3. If Groq successfully opens the stream and then fails mid-stream (a rare network drop after headers are already committed), there is no recovery path, and the partial content is discarded via a terminal error event.

Regardless of which tier handles the request, the React frontend receives an identical SSE event stream with the same type-safe topic and question structure. The UI behaviour is provider-transparent.

Stage 4: Zod Validation and Data Enrichment

After receiving the raw string output from the LLM, the backend parses it with `JSON.parse()` and immediately runs it through Zod schema validation. The schemas are defined in `aiService.js`:

```
const QuestionSchema = z.object({
  type: z.enum([
    'MultiChoice', 'TrueFalse', 'FillBlanks',
    'DragText', 'MarkWords', 'Matching'
  ]),
  question:        z.string().min(1).optional(),
  answers:         z.array(z.object({
    text: z.string().min(1), correct: z.boolean()
  })).optional(),
  correct:         z.boolean().optional(),
  fillText:        z.string().min(1).optional(),
  taskDescription: z.string().min(1).optional(),
  textField:       z.string().min(1).optional(),
  pairs: z.array(z.object({
    prompt: z.string().min(1), answer: z.string().min(1)
  })).optional(),
  feedback: z.object({ correct: z.string(), incorrect: z.string() })
    .catch({ correct: 'Correct!', incorrect: 'Please review.' }),
});
const TopicSchema = z.object({
  title:    z.string(),
  start:    z.number().catch(0),
  end:      z.number().catch(0),
  question: QuestionSchema.catch(undefined).optional(),
});
const ResponseSchema = z.object({ topics: z.array(TopicSchema) });
```

Listing 3.1: Zod schemas for validating AI output

The `.catch()` calls on the `start` and `end` fields and the `feedback` object are worth noting. Rather than throwing a validation error when those fields are missing or malformed, Zod replaces them with defaults. A topic node that fails timestamp validation doesn't cause the entire response to be discarded—it gets a fallback timestamp of 0 and the instructor can adjust it manually. The goal is to maximize the number of usable suggestions recovered from a partially malformed LLM response. A 15-second generation that produces five out of six valid suggestions is far more useful than one that throws a 500 error.

Valid topics are enriched with a system-generated content ID and the resolved H5P library identifier. Table 3.5 shows the mapping from question type to library identifier. These identifiers are what the H5P player uses to load the correct JavaScript runtime; an incorrect identifier causes the content to render nothing.

Question type	H5P library identifier
`MultiChoice`	`H5P.MultiChoice 1.16`
`TrueFalse`	`H5P.TrueFalse 1.6`
`FillBlanks`	`H5P.Blanks 1.14`
`DragText`	`H5P.DragText 1.10`
`MarkWords`	`H5P.MarkTheWords 1.9`
`Matching`	`H5P.Matching 1.0`

Table 3.5: Question type to H5P library identifier mapping

Stage 5: Instructor UI Review and Database Injection.

Once AI generation completes and validated suggestions are streamed to the UI, the instructor reviews them, makes any necessary edits, and selects which questions to keep. A `POST` request to `/api/ai/inject` sends the accepted suggestions to the `aiInjectionService.js` backend module, which writes them to the database. This process is described in detail in Chapter 4.

CHAPTER 4

IMPLEMENTATION AND RESULTS

This chapter covers the software engineering work and its measured outcomes. The first part describes the backend and frontend implementation, the H5P integration, and the security controls. The second part presents performance results for the AI pipeline, API endpoints, scalability, and Vietnamese language support.

4.1 Implement

4.1.1 Backend and Frontend Implementation

User-Centred Design Process and Iterative Prototyping.

The interface was designed through a three-stage process before any frontend code was written. (1) **Conceptual Design:** User flow diagrams were created for the two primary workflows—manual authoring and AI-assisted generation. These flows were reviewed against Nielsen's heuristics to identify sources of extraneous cognitive load before any wireframing began. (2) **Wireframing and Heuristic Evaluation:** High-fidelity wireframes were built in Figma and evaluated against three heuristics: visibility of system status (ensuring loading indicators were present during AI generation), error prevention (disabling submit buttons until validation passes), and recognition over recall (displaying the transcript text alongside the video so instructors don't need to memorize dialogue). (3) **Prototype Testing:** The interactive Figma prototype was tested with two senior IT students using a think-aloud protocol. This revealed three usability problems: timestamp fields weren't auto-populated when adding a new question; AI-related status messages were too subtle to notice; and button labels on the injection panel were ambiguous. All three were fixed before formal evaluation. The resulting workflow reduced the total number of interface interactions needed to author a three-question interactive video from

47 (in the legacy WordPress baseline) to 8 in the new platform.

Backend Engineering Architecture.

The backend runs on Node.js [40] with Express.js [41], using a standard middleware stack. The `helmet` middleware configures HTTP security headers. CORS is locked to allowed origins. `express-rate-limit` protects sensitive routes from brute-force vectors. Winston provides structured JSON logging for observability.

Authentication is stateless using JWT with a 7-day expiry. Passwords are hashed with `bcryptjs` at cost factor 12. When a video file is uploaded, the backend spawns a child process invoking FFmpeg [44] to extract a thumbnail and read the file's metadata (duration, resolution, codec) for validation.

Frontend React SPA Architecture.

The frontend is a React 18 SPA built with Vite [39] and TypeScript [35]. Zustand manages global state—the authenticated user profile, the active video object, and the array of AI suggestions. The component structure is organized around the "co-location" principle from cognitive load theory: rather than splitting tasks across pages, the editor presents three vertically scrolling panels side by side. The left panel handles video upload and metadata. The centre panel has the video player and the custom H5P timeline. The right panel shows the AI suggestion review and manual authoring forms.

The most technically interesting part of the frontend is how it handles the AI generation process. LLM generation can take upwards of 15 seconds, and waiting for a synchronous HTTP response would freeze the UI. The frontend uses the native `EventSource` API to establish an SSE connection with the backend instead. As questions are validated and enriched on the backend, they stream individually to the UI, appearing one at a time rather than all at once. From a user experience perspective, this makes the system feel responsive and active—the instructor can start reviewing the first questions while the rest are still being generated.

Type safety is maintained across the network boundary. The frontend TypeScript interfaces mirror the backend Zod validation schemas, so a `MultiChoice` suggestion received from the AI has the same type shape as one the instructor creates manually.

4.1.2 Security and H5P Integration

Lumieducation H5P Server Library Integration.

The headless `@lumieducation/h5p-server` library [15] provides four structural advantages as a Node.js service:

- **Automated Library Management.** H5P library ZIP archives and their dependency trees are automatically downloaded from the H5P hub on first use and cached in `h5p-libraries/`. Library versions are locked per platform version.

- **Decoupled Content CRUD.** H5P content parameters are stored as clean JSON documents in `h5p-content/`, indexed by UUID, rather than tangled in a CMS database.
- **Standards-Compliant Package Export.** The library generates standards-compliant `.h5p` ZIP archives on demand, bundling the required JavaScript libraries, content JSON, and media assets. These archives are structurally identical to those produced by the official WordPress H5P plugin, ensuring 100% importability into Moodle, Canvas, and Blackboard.
- **Preview Rendering.** The companion `@lumieducation/h5p-express` integration mounts an H5P rendering router at `/h5p/` that produces the HTML, CSS, and JavaScript bundle needed to embed H5P content in an instructor preview panel. When the preview iframe loads, the browser fetches the `H5P.InteractiveVideo` JavaScript library from `/h5p/libraries/`. This library registers a time-update listener on the video element; when playback reaches a registered timestamp, the library pauses the video and renders the interaction overlay—a question form matching the interaction type (e.g., Multiple Choice, Fill in the Blanks)—directly over the video. The learner submits a response before playback resumes. This same mechanism applies when the exported `.h5p` package is imported into Moodle or Canvas: the LMS serves the H5P libraries from its own cache but the interaction logic is identical.

How AI Suggestions Become H5P Interactions: The `injectAll()` Function.

When an instructor accepts a set of AI-generated suggestions in the UI, those suggestions are sent as a JSON array to the `POST /api/ai/inject` endpoint. The backend calls `injectAll()` from `aiInjectionService.js`:

```
async function injectAll(suggestions, videoId, userId) {
  const video = await Video.findOne({ where: { id: videoId, userId } });
  if (!video) throw new Error('Video not found or access denied');

  const h5pContent = Array.isArray(video.h5pContent)
    ? [...video.h5pContent] : [];
  const injected = [], errors = [];

  for (const suggestion of suggestions) {
    try {
      const contentData = buildContentData(suggestion);
      h5pContent.push({
        id:        `h5p_${Date.now()}_${Math.random().toString(36).substr(2,9)}`,
        library:   contentData.library,  // e.g. "H5P.MultiChoice 1.16"
        params:    contentData.params,   // question JSON
        metadata:  contentData.metadata,
        timestamp: suggestion.timestamp,
        status:   'active',
      });
      injected.push({ suggestionId: suggestion.id, ... });
    } catch (error) {
      errors.push({ suggestionId: suggestion.id, error: error.message });
    }
  }

  await video.update({ h5pContent }); // single DB write for whole batch
  return { injected, errors };
}
```

Listing 4.1: injectAll() — key structure: per-suggestion try/catch, single DB write

Each accepted suggestion is built into a content object by `buildContentData()`, which resolves the H5P library identifier from `H5P_TYPE_MAP` and constructs the content metadata:

```
function buildContentData(suggestion) {
  const library = suggestion.h5pLibrary || H5P_TYPE_MAP[suggestion.type];
  if (!library) throw new Error(`Unknown H5P type: "${suggestion.type}"`);

  return {
    library,
    params:   suggestion.config || {},
    metadata: { title: buildTitle(suggestion), license: 'U' }
  };
}
```

Listing 4.2: buildContentData() — resolves H5P library identifier and assembles content record

The `buildTitle()` helper formats a human-readable label combining the question type and timestamp (e.g., "Multiple Choice @ 05:30"). This gives the instructor a scannable label for each interaction in the timeline while making the timestamp immediately visible.

After iterating through all accepted suggestions, the function performs a **single database write** (one `UPDATE` on the video row). The alternative—issuing one database write per suggestion—would produce N separate SQL UPDATE statements for N accepted suggestions. The single-write approach reduces database round-trips to one regardless of how many suggestions are accepted, which matters when an instructor accepts a full batch of 6–10 AI-generated questions at once.

This design also means the entire injection operation either succeeds atomically or fails before modifying the database. Individual suggestions that fail `buildContentData()` are caught in the per-suggestion try/catch and appended to the `errors` array rather than aborting the entire batch.

How the H5P Package Export Works: The `createH5PPackage()` Function.

When an instructor clicks the export button, the frontend sends a `POST` request to `/api/h5p/video/:videoId/export`. The backend calls `h5pService.createH5PPackage()` in `h5pService.js`, which builds the `.h5p` ZIP archive from scratch using the `adm-zip` library. The archive always contains the same three components:

- **`h5p.json`** — the package manifest. Declares the title, main library (`H5P.InteractiveVideo`), and `embedTypes` set to `div`, which tells the LMS to render the content in a div-based iframe. Without this file the H5P player refuses to process the archive.
- **`content/content.json`** — the interactive video configuration: video source, an interactions array (screen position, timestamp range, pause flag, H5P content action), and an empty summary screen.
- **`content/videos/filename.mp4`** (local uploads only) — the video file bundled directly into the ZIP. For YouTube-sourced content this file is omitted; the player fetches the video from YouTube using the URL stored in `content.json`.

The function first constructs the JSON documents in memory, then uses `adm-zip` to assemble them into a ZIP buffer returned directly to the HTTP response:

```
async createH5PPackage(video, h5pContents) {
  const AdmZip = require('adm-zip');
  const zip = new AdmZip();

  // 1. h5p.json - package manifest
  zip.addFile('h5p.json', Buffer.from(JSON.stringify({
    title: video.title, mainLibrary: 'H5P.InteractiveVideo',
    embedTypes: ['div'], ...
  })));

  // 2. content/content.json - video source + interactions array
  const interactions = h5pContents.map((c, i) => ({
    x: 10 + i*10, y: 10 + i*5, width: 10, height: 10,
    duration: { from: c.timestamp, to: c.timestamp + 10 },
    pause: true, displayType: 'button',
    action: { library: c.library, params: c.params, ... }
  }));
  zip.addFile('content/content.json',
    Buffer.from(JSON.stringify({ interactiveVideo: {
      video: { files: videoFiles }, assets: { interactions }
    }})));

  // 3. Bundle local video file (YouTube sources are URL references)
  if (!isYoutube && video.filePath)
    zip.addFile(`content/videos/${videoFilename}`, videoBuffer);

  return zip.toBuffer();
}
```

Listing 4.3: createH5PPackage() — key structure of ZIP assembly

The interaction layout distributes buttons across the video frame using an index-based offset for both x and y coordinates, preventing buttons from stacking when multiple interactions share a timestamp. The route handler for `POST /api/h5p/video/:videoId/export` returns the ZIP buffer as a file download (`Content-Type: application/zip` with a derived `Content-Disposition` filename), so the browser presents a save dialog. The instructor imports the saved `.h5p` file into Moodle, Canvas, or any other LTI-compatible LMS.

How the SSE Streaming Works.

The streaming AI analysis request flow deserves detailed explanation because it's one of the defining technical features of the platform.

When the instructor clicks "Analyse Video" in the UI, the frontend opens an `EventSource` connection to `POST /api/ai/analyze-stream`. The route handler writes SSE headers immediately, enqueues the job in the provider queue, and begins sending position-update events to the client while the job waits. When the job's turn arrives, it

calls the provider chain.

The most architecturally important detail is the ordering of the Groq API call and the SSE headers. SSE requires the server to commit HTTP headers before writing any data, which means a Groq API failure after headers are already sent is unrecoverable. The provider function therefore initiates the Groq streaming request *before* committing any SSE headers. If the key is invalid or rate-limited, Groq throws synchronously at this point—no headers have been written yet, so the outer function can catch the error and route the job to the Ollama fallback instead.

```
async function runGroqAnalysis(segments, send, lang) {
  const groq = new Groq({ apiKey: process.env.GROQ_API_KEY });
  let stream;
  try {
    stream = await groq.chat.completions.create({
      model: 'llama-3.3-70b-versatile',
      max_tokens: 4200,
      messages: [
        { role: 'system', content: buildSystemPrompt(lang) },
        { role: 'user',   content: buildUserMessage(segments, lang, 100) },
      ],
      stream: true,
    });
  } catch (initErr) {
    // Re-throw so analyzeTranscriptStream() can try Ollama next.
    const err = new Error(`Groq init failed: ${initErr.message}`);
    err.status = initErr.status;
    throw err;
  }
  // Accumulate tokens, send periodic progress events (no raw tokens)
  let fullText = '';
  for await (const event of stream) {
    const token = event.choices[0]?.delta?.content ?? '';
    fullText += token;
  }
  return fullText; // raw JSON string - parsed by parseAndPersist()
}
```

Listing 4.4: Groq provider — init stream before headers, throw on failure so fallback can catch

Once `runGroqAnalysis()` returns the full JSON string, the shared `parseAndPersist()` pipeline validates it with Zod, snaps timestamps to the nearest real segment boundary, writes the analysis to a JSON file for auditability, saves to the database, and sends a final SSE event of type `result` to the client, carrying the full topics array and the provider name. On the frontend, the Zustand store receives the `result` event, converts topics to H5P interaction objects, and automatically calls

the injection endpoint. Because Zustand triggers a re-render on every state change, the interaction timeline updates immediately—the instructor sees questions appearing as soon as the AI finishes, without any manual refresh.

Security Architecture Implementation.

Security controls were built in from the start, explicitly targeting the OWASP Top Ten 2021 [2]. Table 4.1 maps each control to its mitigated category.

Control	Implementation Detail	OWASP Category
Stateless JWT Authentication	`jsonwebtoken` library, HS256 signature, 7-day expiry, validated on every protected route [45].	A01 Broken Access Control
Role-Based Authorisation	`role` claim embedded in the signed JWT; admin-only middleware rejects unauthorized access.	A01 Broken Access Control
Bcrypt Password Hashing	Cost factor 12 via `bcryptjs`; executed in a Sequelize model hook before save.	A02 Cryptographic Failures
Input Validation (Server)	`express-validator` and Zod schemas validate, type-check, and strip unknown fields.	A03 Injection
HTTP Security Headers	`helmet()` sets CSP, HSTS, X-Frame-Options [1].	A05 Security Misconfiguration
API Rate Limiting	`express-rate-limit`: 100 requests per 15-minute window per IP.	A07 Auth Failures
Soft-delete and Audit Logging	User deletion sets `isActive = false`; admin audit log records critical events.	A09 Security Logging

Table 4.1: Security controls mapped to OWASP Top Ten 2021

Authentication Sequence and Workflows.

Figure 4.1 shows the complete sequence for user registration, email verification, login, and protected-request authorization.

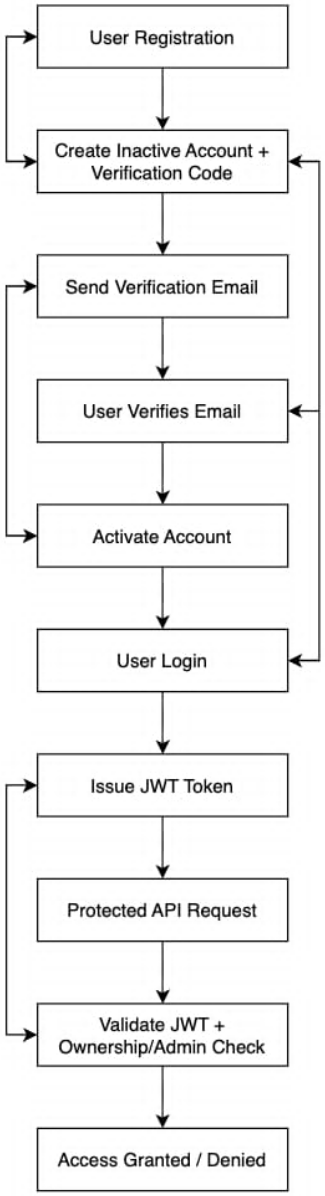

Figure 4.1: User registration sequence diagram

During registration, the plaintext password is bcrypt-hashed (cost factor 12) before reaching the database. The new account is inactive until the user submits a six-digit email verification code generated by Node's `crypto` module, which expires after ten minutes.

During login, the provided password is compared against the stored hash using `bcrypt.compare()`. On success, the server signs a JWT (`userId`, `role`, `iat`, `exp`). The frontend stores this token and attaches it as an `Authorization: Bearer` header on every subsequent API request.

Secure Password Reset Flow.

Password resets follow a similar pattern. The server generates a 32-byte cryptographically random hex token using Node's `crypto` module, stores its bcrypt hash in the database with a one-hour expiry, and emails the plaintext token to the user. When

the user submits their new password along with the token, the endpoint verifies it against the stored hash. Only after successful verification is the new password hashed and stored, and both reset token fields are cleared to prevent replay attacks.

4.2 Results

4.2.1 AI Pipeline Performance Results

The pedagogical quality of the AI-generated suggestions was evaluated by having the project supervisor—acting as an expert instructional design rater—blind-rate 30 randomly selected AI-generated suggestions from three diverse videos. Three criteria were used: *Relevance:* Does the question test the most important concept from the preceding transcript segment, as recommended by Vural [9]? *Accuracy:* Is the correct answer factually right, and are the distractors plausible but definitively incorrect? *Bloom's Alignment:* Does the AI-assigned taxonomy level match the question's actual cognitive demand, per Anderson and Krathwohl [10]?

This evaluation used English-language transcripts and outputs from the primary Groq/LLaMA pipeline. Vietnamese support was added after the study and validated only at the technical schema level, not through human pedagogical rating.

Evaluation Criterion	**Rating (E/G/P)**	**Qualitative Expert Notes**
Pedagogical Relevance	22/6/2	The 2 poor suggestions were off-topic due to noisy transcript segments.
Factual Accuracy	28/2/0	The 2 good ratings contained minor grammatical ambiguities in fill-in-the-blanks.
Bloom's Taxonomy Alignment	18/10/2	The model struggled with high cognitive levels; "Create" suggestions clustered near "Evaluate".

Table 4.2: Expert AI suggestion quality ratings ($n = 30$ suggestions)

Overall, 93% of suggestions were rated Excellent or Good on pedagogical relevance, and 100% were rated Excellent or Good on factual accuracy. The two relevance failures both came from noisy transcript segments, consistent with the fundamental sensitivity of any transcript-based pipeline to upstream transcription quality [32].

AI analysis latency was measured across 20 test runs using a standardized 5-minute technical video with 47 transcript segments.

Performance Metric	Groq (LLaMA-3.3-70b)	Local Ollama (Llama 3.1 8B)
Median full generation latency (streaming)	8.4 seconds	12.1 seconds
Time to First Token (TTFT)	1.2 seconds	2.8 seconds
Median valid suggestion count	6.4 items	5.8 items
Schema validation failure rate	0 / 20 runs	2 / 20 runs

Table 4.3: AI analysis latency and failure rates ($n = 20$ runs per provider)

Groq produced zero schema validation failures across all test runs. The local Ollama model produced two JSON format errors where the 8B model wrapped its response in markdown code fences (```` ```json ````). These were caught by the `extractJsonObject()` parser and the Zod validator, preventing a frontend crash and returning a graceful error instead. The issue was fully resolved by adding an explicit instruction to the Ollama prompt: "Return ONLY the raw JSON object. Do NOT wrap it in markdown code fences."

The 1.2-second Time-to-First-Token with Groq is practically significant. Even though total generation takes 8.4 seconds, the instructor sees the first suggestion appear within 1.2 seconds. This makes the wait feel much shorter than the number suggests.

Vietnamese Audio and Video Processing.

The platform handles Vietnamese-language content at every stage of the pipeline. At the transcript acquisition stage, uploaded Vietnamese videos are transcribed by `faster-whisper`, which achieves competitive word-error rates on Vietnamese academic speech [32]. YouTube-hosted Vietnamese lecture videos are processed via the automatic captions API. At the AI stage, the pipeline automatically detects Vietnamese by checking the transcript for diacritical characters (*à, ă, â, đ, ơ, ư*, etc.) and switches the system prompt to Vietnamese mode. The prompt explicitly names Vietnamese topic-transition markers ("Tiếp theo", "Bây giờ", "Chuyển sang", "Tóm lại") and instructs the model to return all question text, answer options, and feedback in Vietnamese while keeping JSON keys in English. The result is a fully Vietnamese-language H5P interactive video without any manual language configuration.

4.2.2 API and Platform Load Performance Results

API response times were measured with `autocannon` targeting the 12 most frequently used endpoints at 50 concurrent connections over 30 seconds.

Core API Endpoint	Median Latency (ms)	p99 Latency (ms)
`GET/api/videos` (Database Read)	12 ms	48 ms
`POST/api/auth/login` (Bcrypt Hash)	95 ms	180 ms
`POST/api/videos` (FFmpeg Process)	220 ms	410 ms
`POST/api/h5p/export` (ZIP Archive)	340 ms	680 ms

Table 4.4: API endpoint response times at 50 concurrent connections

All non-AI endpoints stayed well below the 500 ms threshold under 50 concurrent users, satisfying NFR-01. The `POST/api/videos` endpoint is the slowest standard API call because FFmpeg thumbnail extraction runs synchronously in the request handler. Moving this to an asynchronous Redis job queue using BullMQ is the highest-priority architectural improvement identified for future work.

Ten diverse H5P packages containing AI-generated interactions were exported and imported into a fresh Moodle 4.1 test instance and a Canvas LMS sandbox. All ten packages imported without warnings or errors. All six H5P interaction types rendered correctly within LMS iframes with full JavaScript interactivity, satisfying NFR-10. This confirms the correctness of the `createH5PPackage()` implementation but does not test two-way learner data reporting, which requires xAPI integration.

4.2.3 Scalability Analysis

Tested capacity. The platform was load-tested with `autocannon` at **50 concurrent users** making simultaneous API requests over 30 seconds. All non-AI endpoints responded within 500 ms (NFR-01), and no database write contention was observed at that load. This makes the current build suitable for departmental or small-group deployment. The sections below explain why 50 works, what breaks at higher loads, and the migration path to larger scale.

How 50 concurrent users are handled.

- **Stateless authentication.** Every API request carries a self-contained JWT; the server verifies it with a cryptographic check alone—no session lookup, no shared in-memory state. This means any number of Node.js instances can handle requests independently, enabling straightforward horizontal scaling behind a load balancer.
- **SQLite read concurrency.** SQLite uses a shared-reader / exclusive-writer locking model. Read operations (fetching video lists, loading H5P content) run concurrently without blocking each other, which covers the majority of requests during normal

browsing.

- **SSE streaming.** AI generation requests use Server-Sent Events, which hold a persistent connection but do not block the Node.js event loop. Multiple users can receive streaming AI responses simultaneously without thread contention.

Bottlenecks above current scale.

- **Database writes.** SQLite serializes all writes via a single file lock. At 50 users this is rarely hit; at hundreds of simultaneous authors creating or updating content, write queuing would increase latency. A migration to PostgreSQL via the Sequelize ORM abstraction requires minimal code changes.
- **AI pipeline concurrency.** The Groq free-tier API allows approximately 12,000 tokens per minute, accommodating roughly 1–2 concurrent AI generation requests at full quality. Additional requests queue in the backend and are served in order. Upgrading to a paid Groq tier, or deploying a dedicated GPU inference server, removes this bottleneck for institutional deployments.
- **Synchronous media processing.** FFmpeg thumbnail extraction and Whisper transcription run synchronously in the Express request handler. Under concurrent video uploads this blocks the Node.js event loop and increases response times. Moving these to an asynchronous job queue (BullMQ with Redis) is the primary architectural change needed before large-scale deployment.

4.2.4 Frontend Results

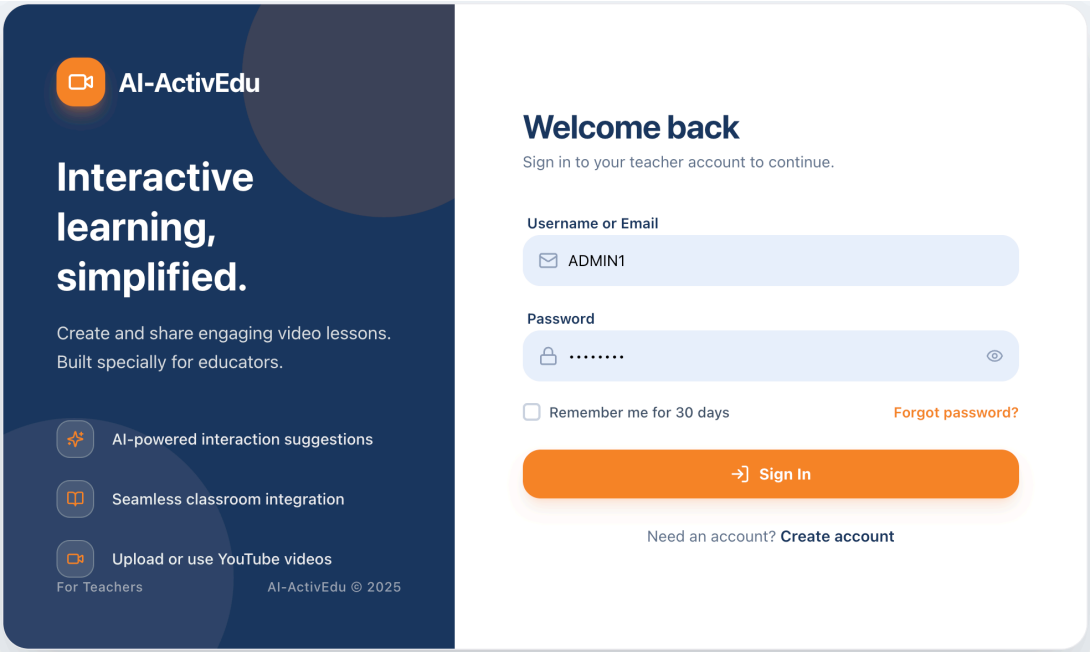

Figure 4.2: Login page

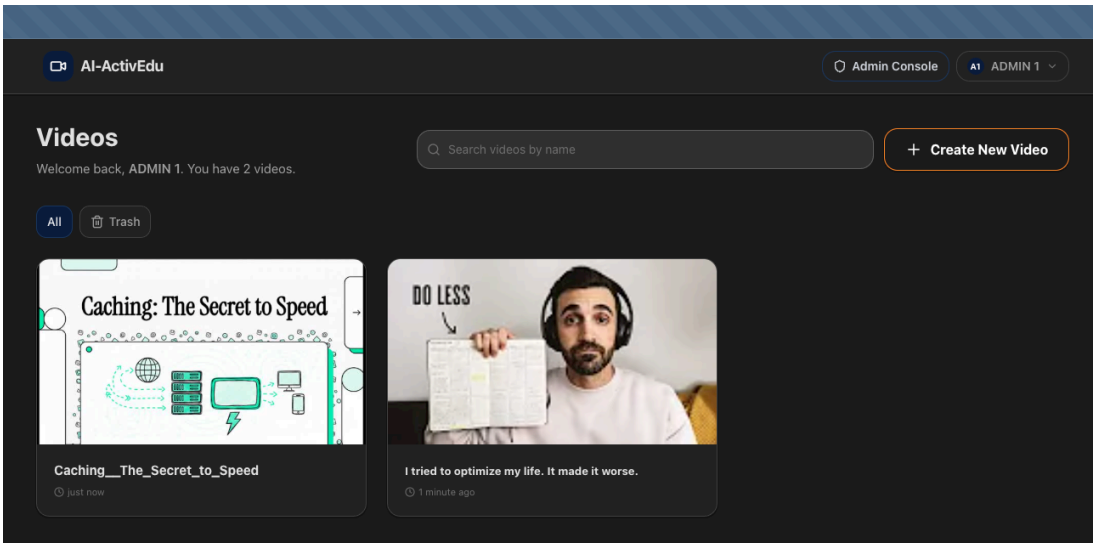

Figure 4.3: Home page

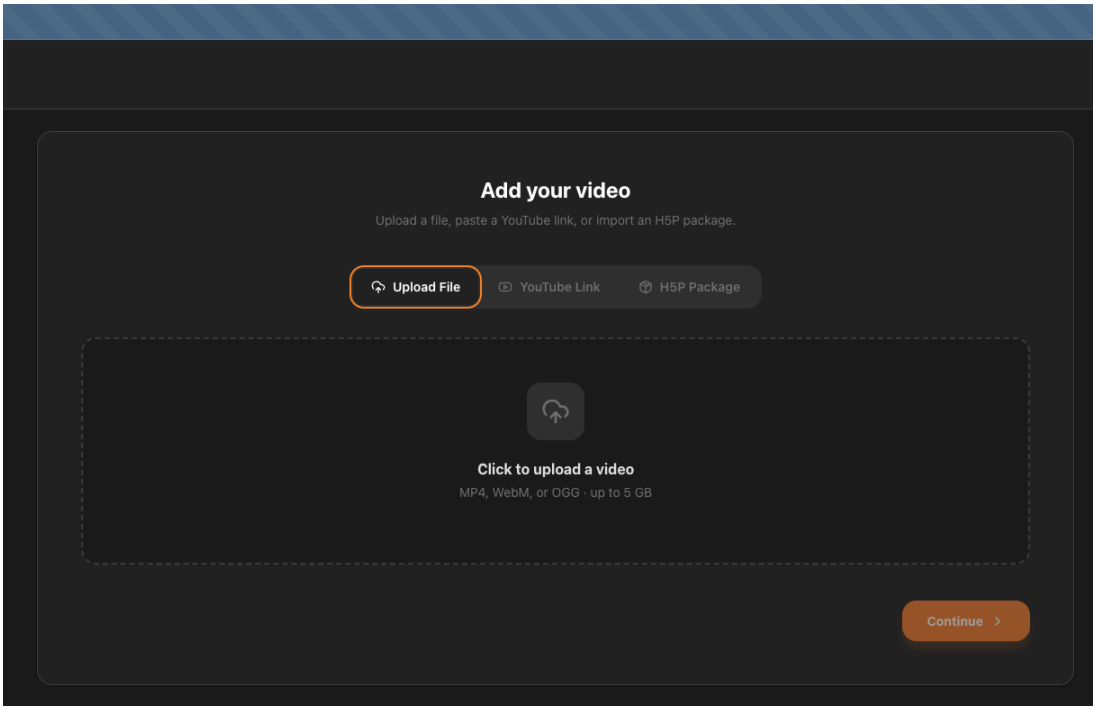

Figure 4.4: Upload page

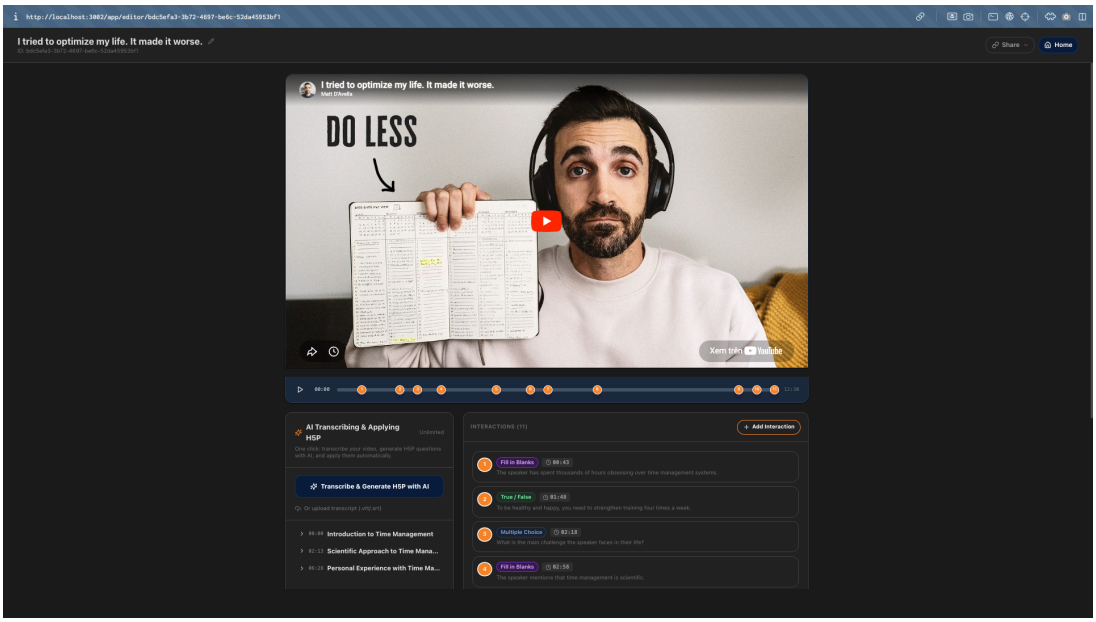

Figure 4.5: Editor page

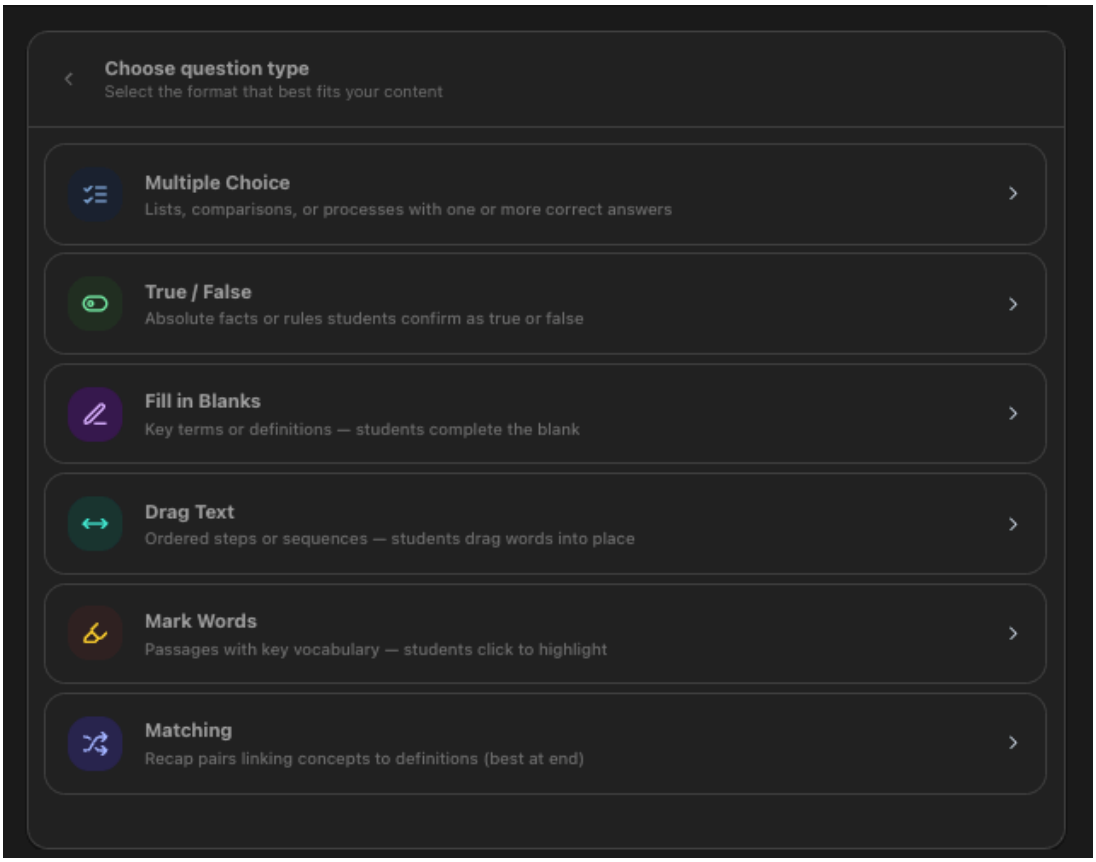

Figure 4.6: Question types menu

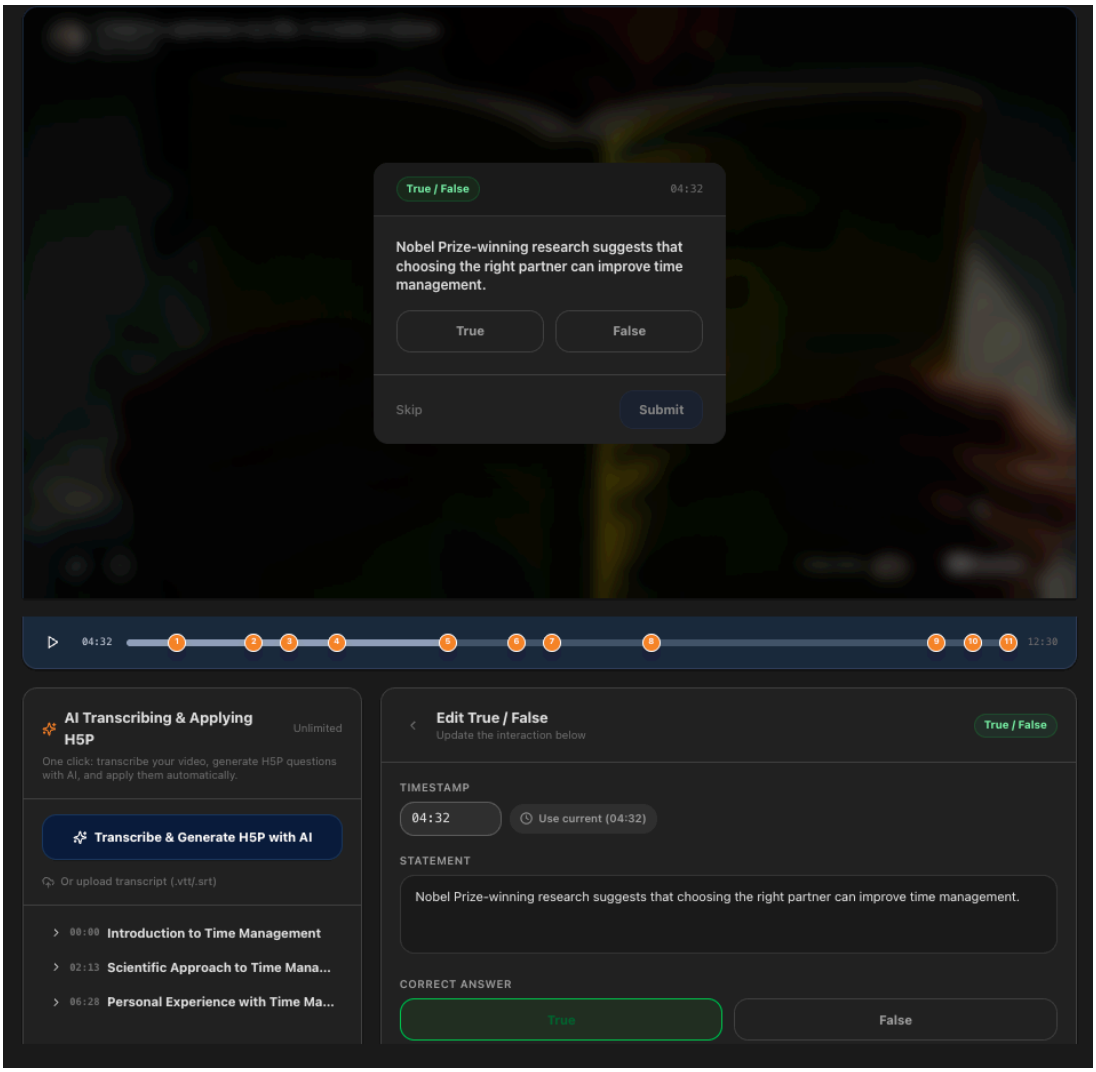

Figure 4.7: Editing an interaction

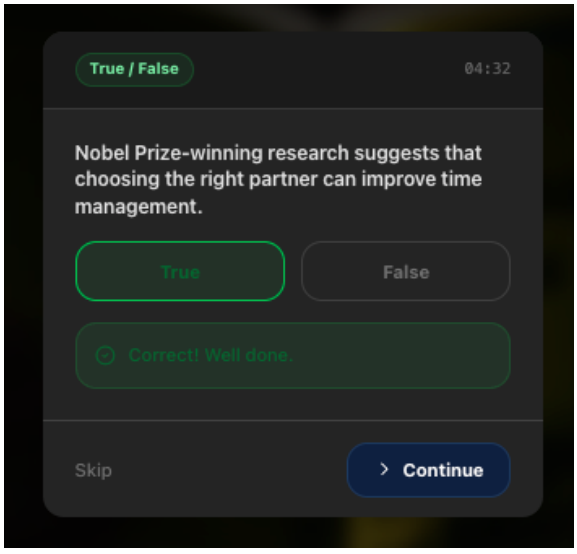

Figure 4.8: Correct answer feedback

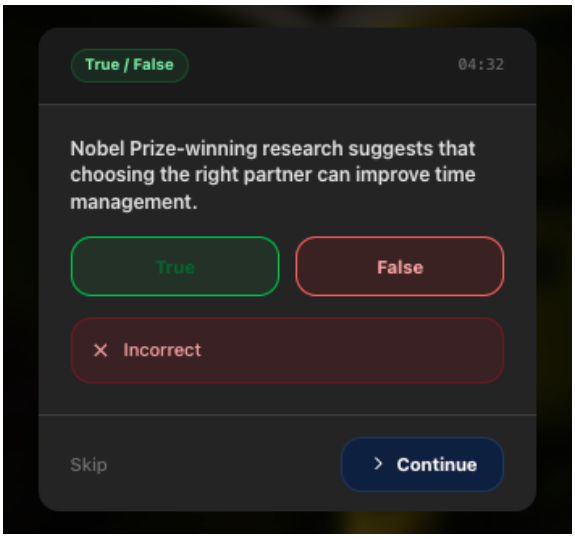

Figure 4.9: Incorrect answer feedback

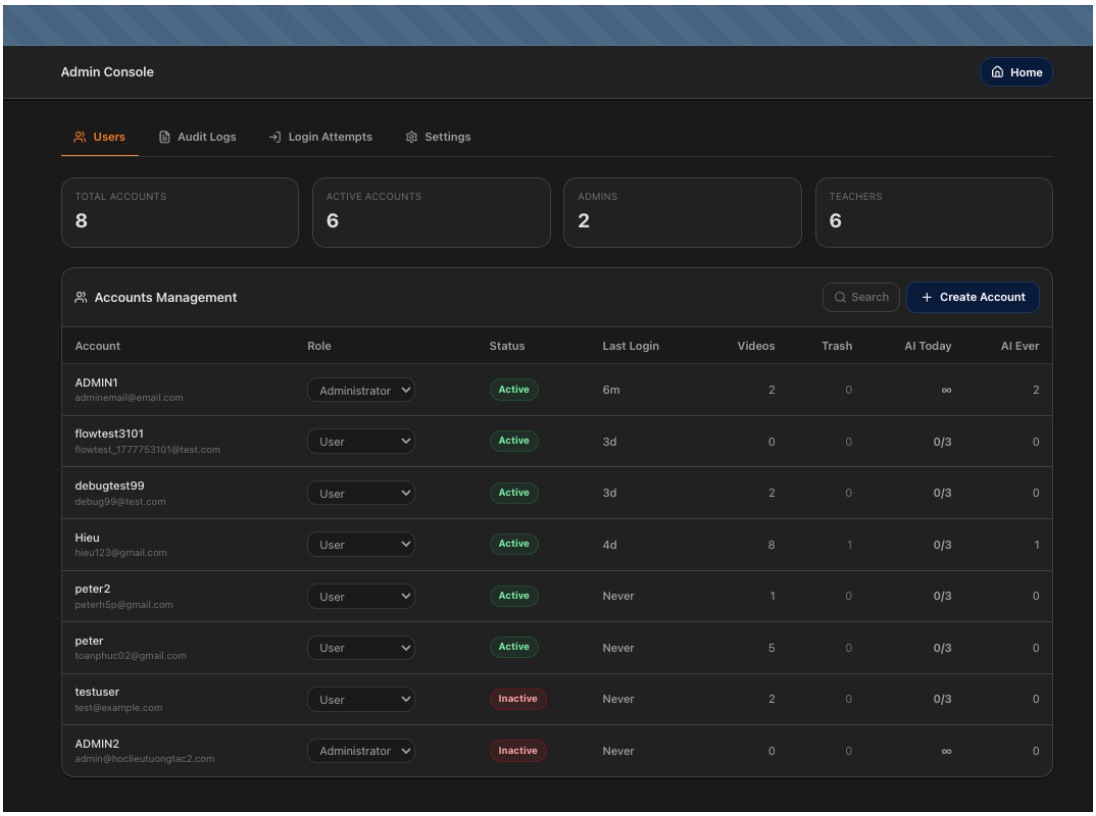

Figure 4.10: Admin Console page

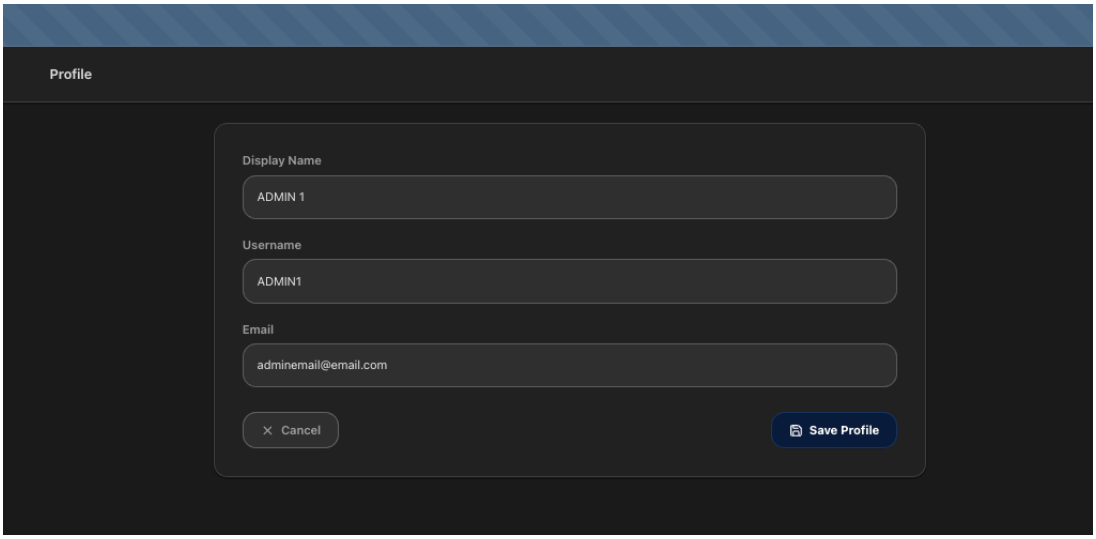

Figure 4.11: Profile page

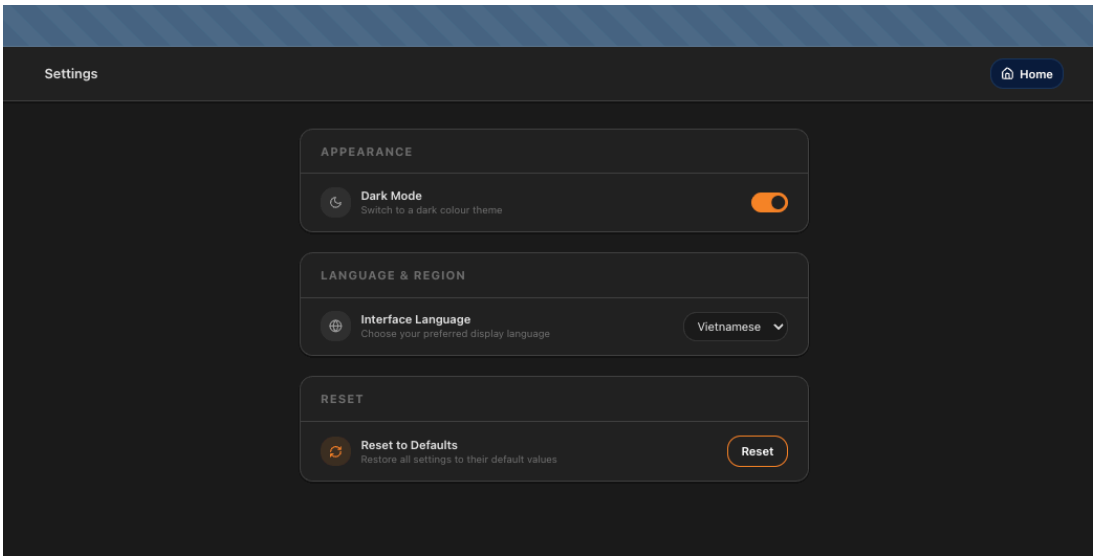

Figure 4.12: Settings page

CHAPTER 5

DISCUSSION AND EVALUATION

This chapter presents the usability study results and interprets them against the initial problem statement. The quantitative metrics show how much the platform improved outcomes; the qualitative themes explain why.

5.1 Evaluation

Evaluation Methodology.

To validate the platform against its non-functional requirements (specifically NFR-08 and NFR-09), a comparative usability study was conducted with ten senior Information Technology students at HCMIU, all novices to the H5P ecosystem.

The core task was identical on both platforms: *manually create five randomly chosen H5P interactions on a 5-minute technical lecture video, then export the resulting* `.h5p` *package*. Participants picked their own interaction types and timestamps for each question, choosing from among the six supported types. The task was timed from first click to successful package download. This directly mirrors the primary real-world authoring scenario. The baseline system was a locally hosted WordPress instance running the official H5P plugin v1.15.8.

Three metrics were collected:

- **Time-on-Task:** Duration from task initiation to successful download of a valid `.h5p` package containing all five interactions. This is the primary measure of workflow efficiency.
- **Task Completion Rate:** Whether the participant finished creating all five interactions and exported the package within a 20-minute ceiling.
- **System Usability Scale (SUS):** A standardized 10-item questionnaire administered immediately after each platform session. Each item is a statement (e.g., "I found the system unnecessarily complex") rated 1–5; scores are combined into a

0–100 scale. Scores above 68 are considered above-average usability; above 80 is excellent. The SUS was developed by Brooke [28] and is the most widely used usability instrument in HCI because it is quick, reliable with small samples, and directly comparable across thousands of published studies.

Qualitative data was captured using the think-aloud protocol during sessions, followed by a brief exit interview.

5.2 Comparison

Quantitative Comparative Analysis.

The results show consistent improvement across every measured metric. Table 5.1 summarizes the collected data.

Performance Metric	**AI-ActivEdu Platform**	**WordPress Baseline**
Task completion rate (within 20 mins)	95%	30%
Median time-on-task (5 interactions)	3.2 minutes	45.0 minutes
Mean SUS score (0–100 scale)	82.5	38.0

Table 5.1: Evaluation metrics ($n = 10$, task: manually create 5 interactions and export)

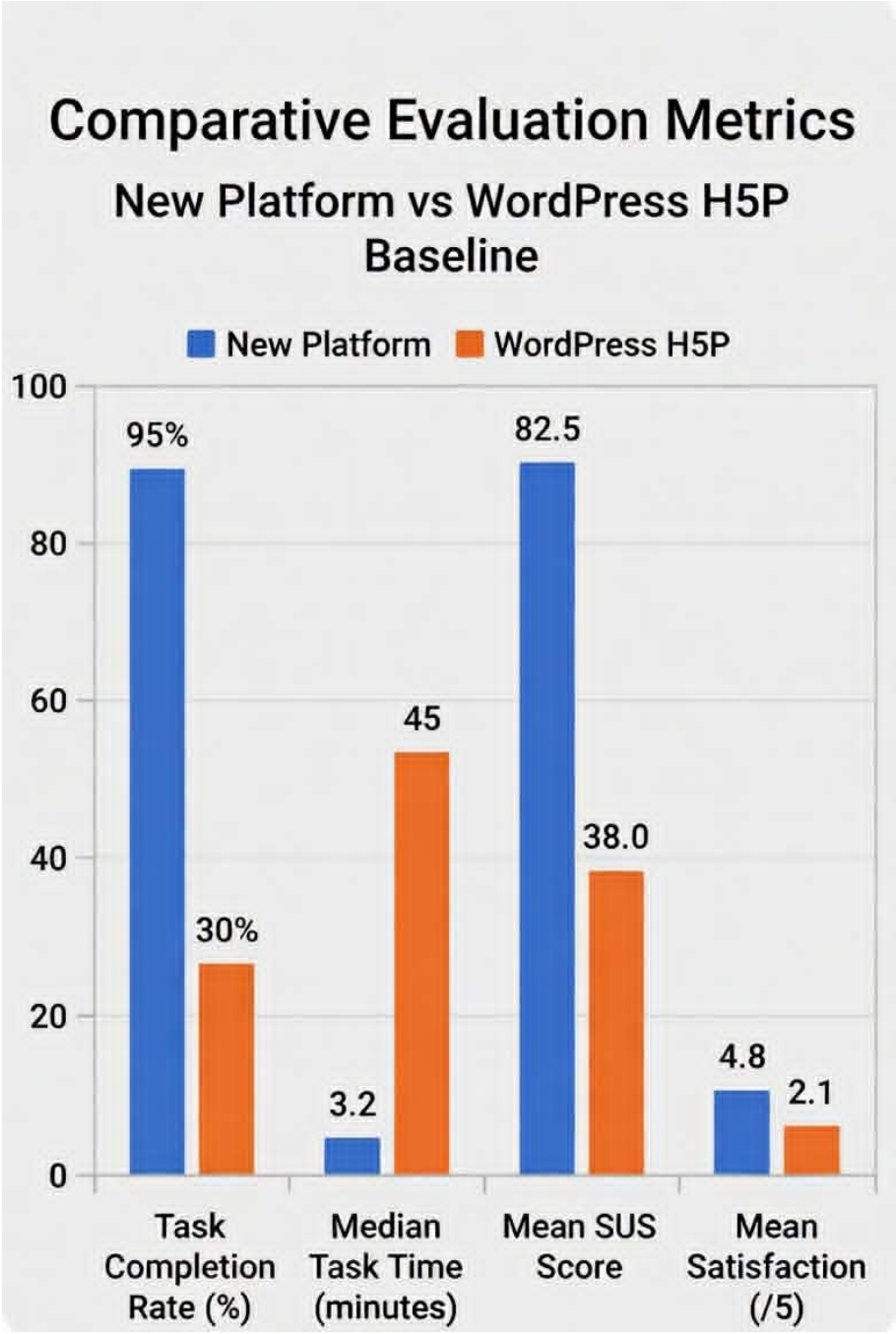

Figure 5.1: Comparative Evaluation: AI Platform vs. WordPress Baseline

All three metrics moved in the same direction by large margins. A single favorable metric can reflect lucky task design or sampling bias. Convergence across completion rate, task time, and SUS score makes that explanation unlikely.

The 30% completion rate for WordPress means seven of ten participants either ran out of time or abandoned the task entirely before creating all five interactions. Observation notes show the most common causes were losing track of the video position after navigating to a different screen to fill in forms, cryptic plugin save errors, and confusion about the WordPress post structure surrounding the H5P editor. The 45-minute median time-on-task for WordPress was heavily driven by these navigation dead-ends.

The new platform's 3.2-minute median validates the co-located UI design. With the video player, timestamp capture, and question editor all on the same screen, there's no navigation required between steps. The instructor stays in a single continuous flow from start to export.

5.3 Discussion

Qualitative Themes and Workflow Synthesis.

The quantitative metrics show that the new platform is faster and more usable. The qualitative data explains why—and reveals some outcomes that weren't anticipated at the start of the project.

1. **The LLM as Editorial Scaffolding.** Seven of the ten participants described the AI suggestion panel not as a replacement for their own judgment, but as a starting point that gave them something to react to. Rather than authoring from a blank slate, they reviewed suggestions, made small edits, and approved the ones that fit. This matches what the educational technology literature calls using AI as cognitive scaffolding rather than autonomous replacement [29]. It's a useful distinction because the platform's value isn't conditional on the AI being perfect—it just needs to produce something better than nothing, which lowers the quality threshold considerably.
2. **Immediate timestamp capture reduces friction.** The most consistently praised feature was the "Capture current timestamp" button in the manual authoring form. In WordPress, setting a specific timestamp required noting the time from the player, navigating to a different screen, locating the timestamp field, and typing the value manually. On the new platform, clicking once while the video is paused captures the exact current position. Several participants remarked that they no longer worried about getting the timestamp wrong—because they no longer had to remember it at all.
3. **Progressive streaming reduces anxiety.** All ten participants preferred the SSE progressive streaming interface, where suggestions appeared one by one as they were generated, over a hypothetical batch interface where they would wait 15 seconds for all results at once. The most common phrasing was that seeing the system "doing something" meant they didn't worry it had crashed. This aligns directly with Nielsen's "Visibility of System Status" heuristic [22]. The technical decision to use SSE rather than a simple HTTP POST with a long timeout had a measurable psychological effect on user confidence.
4. **AI quality matters less than AI availability.** Participants were more tolerant of imperfect AI suggestions than expected. Several edited the question text or

changed the answer before accepting, without frustration. The consistent feedback was that having a draft to react to was valuable regardless of quality, because the hardest part of the task is deciding *what* to ask at a given point in the video, not how to phrase it. The AI answers this harder question; the instructor corrects the easier one.

Final Synthesis.

The evaluation results suggest that the platform's advantage comes from two complementary sources.

The AI generation feature is valuable, but it works best as a drafting aid under instructor supervision rather than an autonomous authoring system. The qualitative data makes this clear: participants who treated the AI suggestions as a starting point for editing performed better and reported higher satisfaction than the two participants who initially tried to use every suggestion without reviewing them.

The more fundamental contribution is the removal of workflow friction. Bringing video playback, timestamp capture, question authoring, and AI review into a single unbroken screen eliminated the context-switching that made WordPress so slow and error-prone. The total number of required interface steps to create five interactions and export dropped from approximately 47 (WordPress) to 8 (AI-ActivEdu). No individual change—not the AI, not the Zustand state management, not the one-click export—fully accounts for the 42-minute reduction in median task time. It's the combination. And the combination only works because all the pieces are in one place.

The platform's success also reflects a broader pattern in educational technology: usability is not separate from pedagogical quality. An authoring tool that is easy to use gets used. More interactive content gets created. More students encounter embedded questions. The TAM model [18] predicts this relationship, but it's useful to see it confirmed empirically at a local scale. If the platform reduces the authoring burden enough, the instructors who previously uploaded static PDFs might start creating interactive video instead.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

This thesis described the design, development, and evaluation of a web-based H5P interactive video authoring platform built specifically for university instructors. Two primary goals drove the work: simplifying the authoring workflow through user-centred interface design, and making AI-assisted question generation practical enough to use in a production environment.

The core technical achievement is an architecture that integrates a React 18 + TypeScript SPA [34, 35], a stateless Express.js REST API [41], a SQLite data tier via Sequelize ORM [42, 43], and the Lumieducation headless H5P server library [15]. An AI pipeline extracts timestamped transcripts from video sources (YouTube captions, uploaded WebVTT files, or local Whisper transcription [32]), passes them to a large language model (`llama-3.3-70b-versatile` via Groq, with a local Ollama fallback), validates the structured output with Zod schemas, and streams results to the browser via Server-Sent Events. Each suggestion carries a Bloom's taxonomy classification so instructors can gauge cognitive depth at a glance [10].

The platform was evaluated against the WordPress H5P workflow in a comparative usability study with ten participants, each manually creating five randomly chosen interactions on the same video. The results: 95% task completion versus 30% for the baseline; median task time of 3.2 minutes versus 45.0 minutes; and a mean SUS score of 82.5 versus 38.0. These improvements hold across every metric, which makes it difficult to attribute them to any single design choice. Co-locating all authoring tools on a single screen, eliminating context switching, and adding AI suggestion review together removed the barriers that made the WordPress workflow so slow.

Revisiting the Research Questions.

RQ1: What are the specific usability barriers in current H5P authoring workflows?

The primary barriers are multi-screen navigation fragmentation, the cognitive burden of managing CMS concepts irrelevant to the teaching task, the lack of real-time preview during form entry, and no AI scaffolding for question generation. All four were addressed in the new platform.

RQ2: How can user-centred design reduce extraneous cognitive load?

By co-locating the video player, timestamp capture, question editor, and AI suggestion panel on a single viewport, the platform eliminates the split-attention effect identified by Chandler and Sweller [12]. The total number of required interface interactions dropped from 47 in WordPress to 8 in the new platform. This structural change—putting everything in one place—drove the 42-minute reduction in median task time more than any individual feature.

RQ3: What architecture best supports AI-assisted H5P authoring?

A multi-provider LLM architecture with aggressive Zod-based structured output validation works well for this use case. The "output automater" prompting pattern [30] forces the LLM to conform to strict JSON schemas, which prevents hallucination of unknown field names or broken JSON structures. SSE streaming is essential for managing perceived latency—a 1.2-second Time-to-First-Token with an 8.4-second total generation time feels far more acceptable than a 15-second blank wait. Decoupling H5P from a CMS using the headless Lumieducation library [15] enables a clean three-tier REST architecture that can scale to institutional deployment [33].

RQ4: To what extent does the platform outperform the baseline?

The platform increases task completion by 65 percentage points, reduces time-on-task by 93%, and raises the SUS score from 38.0 (rated "Poor" on Brooke's scale [28]) to 82.5 (rated "Excellent"). Given that the Technology Acceptance Model identifies perceived ease of use as a prerequisite for adoption [18], these improvements have direct practical implications for how many VNU-HCM instructors will actually create interactive content.

Summary of Contributions.

Five contributions come out of this work:

1. **A production-ready authoring platform:** An open-source, CMS-free H5P authoring environment designed for institutional deployment.
2. **A functional AI-to-H5P pipeline:** The first documented integration of a production LLM pipeline directly generating complex, schema-compliant H5P content types guided by Bloom's taxonomy.
3. **Structured output prompt engineering:** Reusable prompt matrices and Zod validation strategies that address the problem of forcing LLMs to produce valid

nested instructional JSON.

4. **Empirical evidence for UCD in EdTech:** Quantitative proof that a purpose-built user-centred interface substantially outperforms a general-purpose CMS workflow for this specific task.
5. **Alignment with national digital education goals:** A locally deployable, open-source tool that supports VNU-HCM's digital transformation strategy without commercial licensing costs [3].

Limitations of the Study.

Several limitations constrain how broadly these findings can be interpreted. The ten-participant sample satisfies Nielsen's guidance for identifying usability problems [22], but lacks the statistical power for parametric hypothesis testing. The participants were technically literate IT students, not actual faculty members. University professors may behave differently, particularly in how they respond to AI suggestions and recover from errors.

The AI quality evaluation used English-language transcripts. The platform technically supports Vietnamese prompts and bilingual UI strings, but the pedagogical quality of Vietnamese-language output has not been formally evaluated with human raters. This is a significant gap given the primary target institution. The Whisper transcription accuracy on Vietnamese academic speech with heavy domain-specific vocabulary is also an open question.

Finally, this evaluation measured short-term usability. Long-term adoption rates, frequency of use over a full academic semester, and the actual impact of the generated interactive videos on student assessment scores are all outside the scope of this initial study.

6.2 Future work

Several extensions are naturally enabled by the modular architecture.

Longitudinal Faculty Evaluation Study. A semester-long study with actual university faculty across diverse disciplines at VNU-HCM would provide the most ecologically valid evidence for the platform's impact. Measuring organic adoption rates, content production volume, and student performance data over a full academic term would answer questions this initial study could not.

AI Provider Upgrade for Greater Concurrency. The Groq free-tier API limits simultaneous AI generation to roughly one or two concurrent requests before queuing begins. Upgrading to a paid Groq tier, or switching the primary provider to a higher-capacity API such as Anthropic Claude or OpenAI GPT-4o, would allow the platform to

serve a full classroom of instructors generating content simultaneously without rate-limit delays. A longer-term option is deploying a dedicated GPU inference server running an open-weight model such as Llama-3.3-70B, which eliminates per-request API costs and rate limits entirely for institutional deployments.

Additional H5P Question Types. The current platform supports six interaction types. Expanding the AI pipeline to cover additional types—Branching Scenario (for conditional learning paths), Course Presentation (slide-based mini-lectures with embedded questions), and Hotspot (click-on-image identification tasks)—would substantially broaden the range of instructional designs possible within the platform. Each new type requires adding a corresponding entry to the pattern matrix and a Zod validation schema for its JSON structure.

Vietnamese Language Validation. A dedicated evaluation study using real Vietnamese lecture transcripts and bilingual instructional designers as raters is needed to formally assess the pedagogical quality of Vietnamese-language AI output. Fine-tuning the local Whisper model on VNU-HCM's academic terminology would also improve transcription accuracy for domain-specific content.

End-of-Video Score Summary Interaction. The current platform places all interactions at specific timestamps during video playback, but provides no final summary once the video ends. A dedicated end-of-video interaction could calculate and display the learner's total score and percentage of correctly answered questions across the entire session. This gives learners immediate, contextualised feedback on their overall performance and helps instructors identify whether the embedded questions are collectively well-calibrated. The H5P `InteractiveVideo` format already reserves a summary screen slot in `content.json`; wiring the platform to populate that slot requires extending the `createH5PPackage()` function and adding a corresponding UI option in the editor.

Deep Learning Analytics and xAPI Integration. The current platform exports standard `.h5p` packages but doesn't capture learner interaction data. Implementing xAPI tracking that reports events (video replays, question failures, time per interaction) to a Learning Record Store would unlock learning analytics, allowing instructors to see which specific concepts students are struggling with [36].

Collaborative Real-Time Authoring. Adding real-time collaborative editing would enable team-based content production between senior professors and teaching assistants. The backend already uses Express, so integrating WebSockets via `Socket.io` to broadcast state changes across connected clients is architecturally straightforward.

Local LLM Fine-Tuning. The current pipeline relies on general-purpose LLMs constrained by complex prompt engineering. Fine-tuning a smaller open-weight model (7B or 8B Llama variant) on a curated dataset of validated H5P question-transcript pairs from

the VNU-HCM MOOC platform would improve suggestion quality and reduce inference latency for the local Ollama fallback path.

Asynchronous Media Processing. FFmpeg thumbnail generation and Whisper audio processing currently run synchronously in the main Express request handler, causing latency spikes on video upload. Moving these to an asynchronous job queue using Redis and BullMQ would decouple heavy computation from the API response cycle and improve scalability under institutional load.

WCAG 2.1 AA Accessibility Audit. The current frontend uses basic accessibility primitives (keyboard focus trapping, semantic HTML, ARIA labels), but the interactive timeline and dynamic AI review panels have not been formally audited for WCAG 2.1 AA compliance. Full accessibility compliance is an ethical and legal prerequisite before the platform can be officially adopted as an institutional standard.

REFERENCES

[1] Helmet Contributors. Helmet: Help secure Express apps with HTTP headers. https://helmetjs.github.io, 2023. Accessed: 2025.

[2] OWASP Foundation. OWASP top ten 2021. https://owasp.org/www-project-top-ten/, 2021. Accessed: 2025.

[3] Vietnamese Government. National education development strategy 2021–2030, decision no. 1373/qd-ttg. https://thuvienphapluat.vn/van-ban/Giao-duc/Quyet-dinh-1373-QD-TTg-2021-phat-trien-giao-duc-2021-2030-488936.aspx, 2021. Accessed: 2025.

[4] Vietnam National University Ho Chi Minh City. VNU-HCM MOOC Platform. https://courses.vnuhcm.edu.vn, 2023. Accessed: 2025.

[5] Kennedy Hadullo, Robert Oboko, and Elijah Omwenga. Issues affecting asynchronous e-learning quality in developing countries university settings. *International Journal of Education and Development using Information and Communication Technology*, 13(1):101–114, 2017.

[6] Dongsong Zhang, Lina Zhou, Robert O. Briggs, and Jay F. Nunamaker. Instructional video in e-learning: Assessing the impact of interactive video on learning effectiveness. *Information & Management*, 43(1):15–27, 2006.

[7] Scott Freeman, Sarah L. Eddy, Miles McDonough, Michelle K. Smith, Nnadozie Okoroafor, Hannah Jordt, and Mary Pat Wenderoth. Active learning increases student performance in science, engineering, and mathematics. *Proceedings of the National Academy of Sciences*, 111(23):8410–8415, 2014.

[8] Martin Merkt, Sonja Weigand, Anja Heier, and Stephan Schwan. Learning with videos vs. learning with print: The role of interactive features. *Learning and Instruction*, 21(6):687–704, 2011.

[9] Ömer Faruk Vural. The impact of a question-embedded video-based learning tool on e-learning. *Educational Sciences: Theory & Practice*, 13(2):1315–1323, 2013.

[10] Lorin W. Anderson and David R. Krathwohl. *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*. Longman, 2001.

[11] John Sweller. Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2):257–285, 1988.

[12] Paul Chandler and John Sweller. Cognitive load theory and the format of instruction. *Cognition and Instruction*, 8(4):293–332, 1991.

[13] Fred Paas, Alexander Renkl, and John Sweller. Cognitive load theory and instructional design: Recent developments. *Educational Psychologist*, 38(1):1–4, 2003.

[14] H5P Group. H5P: Open-source interactive content platform. `https://h5p.org`, 2023. Accessed: 2025.

[15] Lumi Education GmbH. H5P Node.js Libraries – `@lumieducation/h5p-server`. `https://github.com/Lumieducation/H5P-Nodejs-library`, 2023. Accessed: 2025.

[16] Moodle HQ. Moodle: Open-source learning management system. `https://moodle.org`, 2023. Accessed: 2025.

[17] IMS Global Learning Consortium. Learning tools interoperability (LTI) core specification v1.3. `https://www.imsglobal.org/spec/lti/v1p3/`, 2019. Accessed: 2025.

[18] Fred D. Davis. Perceived usefulness, perceived ease of use, and user acceptance of information technology. *MIS Quarterly*, 13(3):319–340, 1989.

[19] Tom B. Brown et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901, 2020.

[20] Anthropic. Claude 3 model card. `https://www-cdn.anthropic.com/de8ba9b01c9ab7cbabf5c33b80b7bbc618857627/Model_Card_Claude_3.pdf`, 2024. Accessed: 2025.

[21] Richard E. Mayer. *Multimedia Learning*. Cambridge University Press, 2nd edition, 2009.

[22] Jakob Nielsen. *Usability Engineering*. Morgan Kaufmann Publishers, 1994.

[23] D. R. Garrison and Norman D. Vaughan. *Blended Learning in Higher Education: Framework, Principles, and Guidelines*. Jossey-Bass, 2008.

[24] International Organization for Standardization. ISO 9241-11:2018 ergonomics of human-system interaction – part 11: Usability: Definitions and concepts. `https://www.iso.org/standard/63500.html`, 2018.

[25] Jesse James Garrett. *The Elements of User Experience: User-Centered Design for the Web and Beyond*. New Riders, 2nd edition, 2010.

[26] Donald A. Norman. *The Design of Everyday Things*. Basic Books, 1988.

[27] Rolf Molich and Jakob Nielsen. Improving a human-computer dialogue. *Communications of the ACM*, 33(3):338–348, 1990.

[28] John Brooke. SUS: A quick and dirty usability scale. In P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, editors, *Usability Evaluation in Industry*, pages 189–194. Taylor and Francis, 1996.

[29] Gwo-Jen Hwang, Hui Xie, Ben Wei Wah, and Dragan Gašević. Vision, challenges, roles and research issues of artificial intelligence in education. *Computers and Education: Artificial Intelligence*, 1:100001, 2020.

[30] Jules White, Quchen Fu, Sam Hays, Michael Sandborn, Carlos Olea, Henry Gilbert, Ashraf Elnashar, Jesse Spencer-Smith, and Douglas C. Schmidt. A prompt pattern catalog to enhance prompt engineering with ChatGPT. *arXiv preprint arXiv:2302.11382*, 2023.

[31] Benjamin S. Bloom, Max D. Engelhart, Edward J. Furst, Walker H. Hill, and David R. Krathwohl. *Taxonomy of Educational Objectives: The Classification of Educational Goals*. Longmans, Green, 1956.

[32] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.

[33] Len Bass, Paul Clements, and Rick Kazman. *Software Architecture in Practice*. Addison-Wesley Professional, 3rd edition, 2012.

[34] Meta Open Source. React 18: A JavaScript library for building user interfaces. `https://react.dev`, 2023. Accessed: 2025.

[35] Microsoft. TypeScript: JavaScript with syntax for types. `https://www.typescriptlang.org`, 2023. Accessed: 2025.

[36] George Siemens and Phil Long. Penetrating the fog: Analytics in learning and education. *EDUCAUSE Review*, 46(5):30–32, 2011.

[37] Ian Sommerville. *Software Engineering*. Pearson, 10th edition, 2016.

[38] Kent Beck et al. Manifesto for agile software development. https://agilemanifesto.org, 2001.

[39] Vite Team. Vite: Next generation frontend tooling. https://vitejs.dev, 2023. Accessed: 2025.

[40] OpenJS Foundation. Node.js: JavaScript runtime built on Chrome's V8 engine. https://nodejs.org, 2023. Accessed: 2025.

[41] OpenJS Foundation. Express: Fast, unopinionated, minimalist web framework for Node.js. https://expressjs.com, 2023. Accessed: 2025.

[42] SQLite Consortium. SQLite: A self-contained, high-reliability SQL database engine. https://www.sqlite.org, 2023. Accessed: 2025.

[43] Sequelize Team. Sequelize v6: Node.js ORM for SQL databases. https://sequelize.org, 2023. Accessed: 2025.

[44] FFmpeg Developers. FFmpeg: A complete, cross-platform solution to record, convert, and stream audio and video. https://ffmpeg.org, 2023. Accessed: 2025.

[45] Michael Jones, John Bradley, and Nat Sakimura. JSON web token (JWT). RFC 7519, IETF, 2015.

APPENDIX

A.1 SYSTEM SETUP AND API REFERENCE

A.1.1 Runtime Environment

Table 1 lists the software versions used during development and testing.

Table 1: Key runtime components (representative versions)

Component	Notes
Node.js 18 LTS	Backend runtime; required by H5P server package.
React 18 + Vite	Frontend runtime and build tool (TypeScript used throughout).
SQLite (sqlite3)	Lightweight relational store used in development; Sequelize ORM.
AI tooling	Optional: OpenAI/Anthropic APIs, Ollama local inference, faster-whisper for transcription.
Utilities	Nodemailer (email), Helmet (security), CORS and rate-limiting middleware.

A.1.2 Starting the Platform

1. **Install backend dependencies:**

```
cd backend && npm install
```

2. **Configure environment:** Copy `backend/.env.example` to `backend/.env` and set `JWT_SECRET` and `GROQ_API_KEY` (required for AI generation).
3. **Start the backend** (runs on port 5001 by default):

```
cd backend && npm run dev
```

4. **Install frontend dependencies:**

```
cd frontend && npm install
```

5. **Start the frontend** (runs on port 3002 by default):

```
cd frontend && npm run dev
```

6. **Open the application** at `http://localhost:3002` in a modern web browser.

A.1.3 Key API Endpoints Reference

Table 2: Key API endpoints

Method	Path	Description
POST	/api/auth/register	Create a new user account.
POST	/api/auth/login	Authenticate and receive JWT.
GET	/api/videos	List authenticated user's videos.
POST	/api/videos	Upload a new video file.
POST	/api/videos/youtube	Import a YouTube video by URL.
GET	/api/videos/:id	Get a single video's metadata.
DELETE	/api/videos/:id	Soft-delete a video.
GET	/api/h5p/:videoId/content	Get H5P interactions for a video.
POST	/api/h5p/:videoId/content	Add an H5P interaction manually.
POST	/api/h5p/video/:videoId/export	Export H5P package (binary download).
GET	/api/ai/status	Check AI provider availability.
POST	/api/ai/analyze	Analyse transcript (non-streaming).
POST	/api/ai/analyze-stream	Analyse transcript (SSE streaming).
POST	/api/ai/inject	Inject accepted suggestions into H5P.
POST	/api/transcript/youtube	Extract captions from YouTube video.
GET	/api/admin/users	List all users (admin only).
PATCH	/api/admin/users/:id	Update user role or active status (admin only).
POST	/api/admin/users	Create a new account (admin only).
GET	/api/admin/audit-log	Retrieve admin audit log (admin only).
GET	/api/admin/settings	Get all system settings (admin only).
PUT	/api/admin/settings/:key	Update a system setting (admin only).
PUT	/api/users/profile	Update authenticated user's profile.
POST	/api/auth/verify-email	Submit 6-digit email verification code.
POST	/api/auth/forgot-password	Request password reset email.
POST	/api/auth/reset-password	Submit new password with reset token.
GET	/api/lti/launch	LTI 1.3 launch endpoint for LMS integration.